

We use the same ordering to pack the $h^2 f_{i,j}$ (plus boundary terms, where appropriate) into the vector b . Having decided on an ordering, we can now write the equations (7.1.8) as a matrix equation $Au = b$.

Exercise 7.1.9 Determine the form of the matrix A corresponding to the system (7.1.8) and the ordering of equations and unknowns specified in the previous paragraph. $\square$

Just as in the one-dimensional (ODE) case, the matrix A is very sparse. Since each equation in (7.1.8) involves only five of the unknowns, each row of A has at most five nonzero entries. This is true regardless of the ordering. Again the reason for the sparseness is obvious: each unknown interacts directly only with its four nearest neighbors. In the ordering we have chosen (and in every ordering), A has 4's on its main diagonal. These are the 4's that multiply $u_{i,j}$ in the equation corresponding to the (i,j) th grid point. Each 4 is immediately preceded by and followed by a -1 , corresponding to terms $u_{i,j-1}$ and $u_{i,j+1}$, with exceptions near boundaries. The -1 's corresponding to $u_{i-1,j}$ and $u_{i+1,j}$ appear further out in the row, at a distance of $m-1$ from the main diagonal. This is all rather cumbersome to describe but should be clear to the reader who has worked Exercise 7.1.9. The nonzero entries in A are not packed so tightly around the main diagonal as they were in the ODE case; the semi-band width is m . One might hope to achieve a tight packing by changing the ordering of the unknowns. This would require an ordering that places every grid point directly after two of its nearest neighbors and directly before the other two, which is clearly impossible. The ordering we have chosen is about as good as we are going to get, if minimization of bandwidth is our objective. This is a major difference between the one-dimensional and two-dimensional cases.

The description of A is simplified if we use block matrix notation. The dimension of A is $n \times n$, where $n = (m-1)^2$. We can describe it as an $(m-1) \times (m-1)$ block matrix, where each of the blocks is $(m-1) \times (m-1)$, as follows:

$$A = \begin{bmatrix} T & -I & & & \\ -I & T & -I & & \\ & -I & T & \ddots & \\ & & \ddots & \ddots & -I \\ & & & -I & T & -I \\ & & & & -I & T \end{bmatrix}, \quad (7.1.10)$$

where I denotes the $(m-1) \times (m-1)$ identity matrix, and T is the $(m-1) \times (m-1)$ tridiagonal matrix

$$T = \begin{bmatrix} 4 & -1 & & & \\ -1 & 4 & -1 & & \\ & -1 & 4 & \ddots & \\ & & \ddots & \ddots & -1 \\ & & & -1 & 4 & -1 \\ & & & & -1 & 4 \end{bmatrix}.$$

This way of writing A shows clearly that it is symmetric. It is also nonsingular (Exercise 7.1.16) and even positive definite (Exercise 7.1.17). Thus the system $Au = b$ has a unique solution, which approximates the solution of the PDE.

Now let us consider solving the system $Au = b$. The most obvious difference between the one and two-dimensional cases is that now the matrix dimension is much higher. Now we have $n = (m - 1)^2$, compared with $n = m - 1$ in the one-dimensional case. Thus sheer size can be a problem. For example, if we choose an interval $h = 1/101$, corresponding to $m = 101$, we would have to solve a system of 100 equations in the one-dimensional case or 10,000 equations in the two-dimensional case. Our ultimate objective is to solve a PDE. If our approximate solution is not good enough, we can get a more accurate one by taking a smaller h , that is, a finer grid. This, of course, increases the size of the system $Au = b$. It poses no serious problem in the one-dimensional case. The size of the matrix is $O(m)$, and so is the solution time by banded Gaussian elimination. If we cut h in half, we double the work. The situation is much worse in the two-dimensional case. The size of the matrix is about m^2 , so cutting h in half quadruples the size of the matrix. For example, if we take $h = 1/201$, we have to solve a system of order 40,000.

The size of the system is only part of the problem. There is also the question of bandwidth, which we have already mentioned. In the one-dimensional case, the semi-bandwidth is always two, regardless of how big the system is. In the two-dimensional case, the semi-bandwidth is m , so it increases as we decrease h . Let us consider the cost of solving $Au = b$ by a banded Cholesky decomposition, as described in Section 1.5. There we noted that the flop count for solving a system of dimension n and semi-bandwidth s is about $ns^2/2$. Here we have $n = (m - 1)^2$ and $s = m$, so $ns^2/2 \approx \frac{1}{2}m^4$. Thus we see that each time we halve h (doubling m), we multiply the amount of computational work by a factor of sixteen.

Storage space is also a consideration. The matrix A has fewer than $5n$ nonzero entries (≤ 5 per row), so it can be stored very compactly. (Indeed there is no real reason to store A at all; it consists of the numbers 4 and -1 , placed in a regular pattern. If we know the pattern, we know the matrix.) If we want to use Cholesky's method, we have to compute and store the Cholesky factor, which inherits the band structure of A but is not nearly so sparse. Whereas the huge majority of the entries within A 's band are zeros, G 's band is filled in with nonzeros, which have to be stored. The required storage space $ns \approx m^3$. Thus our storage requirement is multiplied by eight each time we halve h .

Are these costs unavoidable, or is there some way around them? It is not hard to derive some basic lower bounds on storage and computational requirements. If the matrix has a very simple form, we may not have to store it at all. However, it seems inevitable that we will have to store the solution u , which consists of n numbers. Thus n is a lower bound on the storage requirement. For our two-dimensional model PDE, $n \approx m^2$. Compared with this figure, the m^3 storage required by Cholesky's method looks like it might be excessive. This turns out to be true. Almost all of

the iterative methods that we will discuss in this chapter require at most a modest multiple of n storage locations.¹

Now let us consider computational costs. Each of the n entries of u has to be computed, and each of these will presumably require at least one flop, so n is also a lower bound on the computational cost. If we solve our model PDE problem by Cholesky's method, the flop count is $\frac{1}{2}m^4 \approx \frac{1}{2}n^2$, so here too it looks like there might be room for improvement. Although it may be too much to hope for a method that needs only some n flops to solve the problem, we might still hope to find one that does the job in Cn flops, where C is some constant. It turns out that the best iterative methods are able to achieve this, at least for highly regular problems like our model problem. The constant C is typically large, so $Cn > \frac{1}{2}n^2$ unless n is fairly large. Once n is big enough, the iterative method will win.

A Three-Dimensional Problem

We live in a three-dimensional world, or so it appears. Hence many applications have three independent variables. The three-dimensional analogue of (7.1.6) is the 3-D Poisson equation

$$-\frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial y^2} - \frac{\partial^2 u}{\partial z^2} = f \quad \text{in } \Omega, \quad (7.1.11)$$

where Ω is a region in $\mathbb{R}^3$. To keep the discussion simple, let us take $\Omega = [0, 1]^3$, the unit cube. Assume boundary conditions of the form (7.1.7).

Discretization of three-dimensional PDE's like this one leads to very large systems of equations.

Exercise 7.1.12 Write down a system of equations analogous to (7.1.8) that approximates the solution of (7.1.11). How many unknowns does your system have? (Assume $h = 1/m$.) How many unknowns appear in each equation? $\square$

Exercise 7.1.13 Consider the system of equations derived in the previous exercise. Suppose the unknowns $u_{i,j,k}$ and the $f_{i,j,k}$ are ordered in a systematic way, and the system is written as a matrix equation $Au = b$. Just as in the one- and two-dimensional cases, A is positive definite and banded. What is the semi-bandwidth of A ? If we wish to solve this system by a banded Cholesky decomposition, how much storage space will be needed? How many flops will it take? $\square$

The previous two exercises demonstrate the difficulties of trying to solve three-dimensional problems by Gaussian elimination. Even with the fairly modest mesh size $h = 1/100$, we get a matrix with dimension $n \approx 10^6$ and semi-bandwidth $s \approx 10^4$. The solution by Cholesky's method takes $\frac{1}{2} \times 10^{14}$ flops and requires the storage of 10^{10} numbers. Each time h is halved, the flop count goes up by a factor of 128, and the storage requirement is multiplied by 32. These numbers show that

¹One notable exception is the popular GMRES method for nonsymmetric systems. See Section 7.9.

we very quickly move into the range where iterative methods have the advantage. Indeed, iterative methods are indispensable to the solution of these problems.

The tendency for computational problems to become much more difficult as the dimension increases is commonly known as *the curse of dimensionality*.

Some Final Remarks

It is not the task of this book to study discretizations of PDE; here we have considered only the most basic technique. For more detailed coverage consult the vast literature, for example [6, 69].

Our model problem is just the simplest of a large class of elliptic boundary value problems. More complicated problems arise when one considers irregularly shaped regions, differential operators with variable coefficients, and other types of boundary conditions. The iterative methods discussed in this chapter can be brought to bear to solve a wide variety of these problems.

One class of techniques for solving the model problem (7.1.8) that will not be discussed in this book is the so-called *fast Poisson solvers*. These are direct methods based on the fast Fourier transform, cyclic reduction, or both. They are nearly optimal for the model problem and other simple problems, but their range of application is limited. Fast Poisson solvers are sometimes incorporated into preconditioners for more complicated problems. An overview of fast Poisson solvers is given in [6].

Additional Exercises

Exercise 7.1.14 In this exercise you will show that the matrix A in (7.1.5) is nonsingular. One characterization of nonsingularity is that if $Aw = 0$, then $w = 0$ as well. Suppose, therefore, that $Aw = 0$.

- (a) Show that the equation $Aw = 0$ is equivalent to the homogeneous system of difference equations

$$-w_{i-1} + 2w_i - w_{i+1} = 0, \quad i = 1, \dots, m-1,$$

with boundary conditions $w_0 = 0$ and $w_m = 0$. Show further that

$$w_i = \frac{w_{i-1} + w_{i+1}}{2}, \quad i = 1, \dots, M-1.$$

In words, each interior value w_i is the average of its neighbors.

- (b) let $\mu = \min\{w_0, \dots, w_m\}$. Prove that if $w_i = \mu$ at some interior point, then $w_{i-1} = w_{i+1} = \mu$ as well. (It is perhaps easier to show the contrapositive: if $w_{i-1} > \mu$ or $w_{i+1} > \mu$, then $w_i > \mu$.) Conclude that $\mu = 0$.
- (c) Let $\mu = \max\{w_0, \dots, w_m\}$. Prove that $\mu = 0$.
- (d) Prove that A is nonsingular.

In working this problem you have developed and used a discrete version of the *maximum principle* for harmonic functions. $\square$

Exercise 7.1.15 In Exercise 5.4.37 you showed that a symmetric matrix is positive definite if and only if all of its eigenvalues are positive. Use this fact, Gerschgorin's Theorem (Exercise 6.5.30), and the result of the previous exercise to prove that the matrix A in (7.1.5) is positive definite. $\square$

Exercise 7.1.16 Use the methodology of Exercise 7.1.14 to prove that the matrix A in (7.1.10) (and (7.1.8)) is nonsingular. $\square$

Exercise 7.1.17 Use the methodology of Exercise 7.1.15 to prove that the matrix A in (7.1.10) (and (7.1.8)) is positive definite. $\square$

7.2 THE CLASSICAL ITERATIVE METHODS

We are interested primarily in sparse matrices, but for the description of the algorithms we can let $A \in \mathbb{R}^{n \times n}$ be (almost) any nonsingular matrix. The minimal requirement is that all of the main diagonal entries a_{ii} be nonzero. Let $b \in \mathbb{R}^n$. Our objective is to solve the linear system $Ax = b$ or $Au = b$. In the early chapters we used x to denote the unknown vector, but in the examples in the previous section we used u . (This change was made because we had another use for the symbol x .) Now we will revert to the use of x for the unknown vector, but we will not hesitate to switch back to u when it seems appropriate.

Iterative methods require an initial guess $x^{(0)}$, a vector in $\mathbb{R}^n$ that approximates the true solution. Once we have $x^{(0)}$, we use it to generate a new guess $x^{(1)}$, which is then used to generate yet another guess $x^{(2)}$, and so on. In this manner we generate a sequence of iterates $(x^{(k)})$ which (we hope) converges to the true solution x .

In practice we will not iterate forever. Once $x^{(k)}$ is sufficiently close to the solution (as indicated, e.g., by the magnitude of $\|b - Ax^{(k)}\|$), we stop and accept $x^{(k)}$ as an adequate approximation to the solution. How soon we stop will depend on how accurate an approximation we need.

The iterative methods that we are going to study do not require a good initial guess. If no good approximation to x is known, we can take $x^{(0)} = 0$. Of course, we should take advantage of a good initial guess if we have one, for then we can get to the solution in fewer iterations than we otherwise would.

The ability to exploit a good initial guess and the possibility of stopping early if only a crude approximant is needed are two important advantages of iterative methods over direct methods like Gaussian elimination. The latter has no way of exploiting a good initial guess. It simply executes a predetermined sequence of operations and delivers the solution at the end. If you stop it early, it gives you nothing.

Jacobi's Method

Each of the methods of this section can be described completely by specifying how a given iterate $x^{(k)}$ is used to generate the next iterate $x^{(k+1)}$. Suppose, therefore, that we have $x^{(k)}$, and consider the following simple idea for improving on it: Use the i th equation to correct the i th unknown. The i th equation in the system $Ax = b$ is

$$\sum_{j=1}^n a_{ij}x_j = b_i,$$

which can be rewritten (solved for x_i) as

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij}x_j \right), \quad (7.2.1)$$

since, as we have assumed, $a_{ii} \neq 0$. If we replace x by $x^{(k)}$ in (7.2.1), equality will no longer hold in general. Let us define $x_i^{(k+1)}$ to be the adjusted value that would make the i th equation true:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij}x_j^{(k)} \right). \quad (7.2.2)$$

Making this adjustment for $i = 1, \dots, n$, we obtain our next iterate $x^{(k+1)}$. This is Jacobi's method.

Of course, $x^{(k+1)}$ is not the exact solution to $Ax = b$, because the correction $x_i^{(k)} \rightarrow x_i^{(k+1)}$, which "fixes" the i th equation, also affects each of the other equations in which the unknown x_i appears. The hope is that repeated application of (7.2.2) for $k = 0, 1, 2, \dots$ will result in convergence to the true solution x .

Example 7.2.3 If we apply Jacobi's method (7.2.2) to the system

$$\begin{bmatrix} 5 & 2 & 1 & 1 \\ 2 & 6 & 2 & 1 \\ 1 & 2 & 7 & 2 \\ 1 & 1 & 2 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 29 \\ 31 \\ 26 \\ 19 \end{bmatrix}$$

with initial guess $x^{(0)} = 0$, we obtain

$$x^{(5)} = \begin{bmatrix} 3.950 \\ 3.074 \\ 2.019 \\ 1.036 \end{bmatrix}, \quad x^{(10)} = \begin{bmatrix} 3.9956 \\ 3.0035 \\ 1.9985 \\ 1.0003 \end{bmatrix}, \quad x^{(15)} = \begin{bmatrix} 3.99972 \\ 3.00026 \\ 1.99992 \\ 1.00005 \end{bmatrix},$$

which steadily approach the exact solution $x = [4, 3, 2, 1]^T$. Any desired accuracy (limited only by roundoff errors) can be obtained by taking enough steps. For example, $x^{(50)}$ agrees with the true solution to twelve decimal places.

Notice that the main-diagonal entries of the coefficient matrix are large relative to the off-diagonal entries. This aids convergence, as the analysis in the next section will show. $\square$

Exercise 7.2.4 Let D denote the diagonal matrix whose main diagonal entries are the same as those of A . Show that equation (7.2.2), which defines Jacobi's method, can be written as a matrix equation

$$x^{(k+1)} = D^{-1} \left[(D - A)x^{(k)} + b \right]. \quad (7.2.5)$$

Show further that

$$x^{(k+1)} = x^{(k)} + D^{-1}r^{(k)},$$

where $r^{(k)} = b - Ax^{(k)}$ is the residual after k iterations. $\square$

Exercise 7.2.6 Write a simple computer program that performs Jacobi iterations (7.2.2), and use it to verify the results shown in Example 7.2.3. Notice how easy this is to program in Fortran, C, or whatever language. A crude MATLAB code based on either of the equations derived in Exercise 7.2.4 is particularly easy. $\square$

Exercise 7.2.7 Use the code you wrote for Exercise 7.2.6 to solve the system

$$\begin{bmatrix} 5 & 1 & 1 \\ 1 & 4 & 1 \\ 1 & 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 10 \\ 12 \\ 12 \end{bmatrix}$$

by Jacobi's method, starting with $x^{(0)} = 0$. Observe the iterates $x^{(k)}$ and the residuals $r^{(k)} = b - Ax^{(k)}$. How many iterations does it take to make $\|r^{(k)}\|_2 < 10^{-5}$? For this value of k , how well does $x^{(k)}$ approximate the true solution? $\square$

It is easy to apply Jacobi's method to the model problem (7.1.8). The equation associated with the (i, j) th grid point is

$$-u_{i-1,j} - u_{i,j-1} + 4u_{i,j} - u_{i,j+1} - u_{i+1,j} = h^2 f_{i,j}.$$

We use this equation to correct the corresponding unknown, $u_{i,j}$. Solving the equation for $u_{i,j}$, we have

$$u_{i,j} = \frac{1}{4} [u_{i-1,j} + u_{i,j-1} + u_{i,j+1} + u_{i+1,j} + h^2 f_{i,j}].$$

Given a current iterate $u^{(k)}$, we obtain a new iterate $u^{(k+1)}$ by

$$u_{i,j}^{(k+1)} = \frac{1}{4} [u_{i-1,j}^{(k)} + u_{i,j-1}^{(k)} + u_{i,j+1}^{(k)} + u_{i+1,j}^{(k)} + h^2 f_{i,j}].$$

As usual, the boundary values of $u^{(k)}$, which are needed in the computation for those grid points that are adjacent to the boundary, are determined by the boundary conditions.

Notice that execution of Jacobi's method does not require writing the system in the form $Au = b$. There is no need to assemble or store the matrix (7.1.10) nor to pack $u_{i,j}$ and $f_{i,j}$ into long vectors.

Example 7.2.8 Jacobi's method was applied to the model problem with $f = 0$ and g given by

$$g(x, y) = \begin{cases} 0 & \text{if } x = 0 \\ y & \text{if } x = 1 \\ (x - 1) \sin x & \text{if } y = 0 \\ x(2 - x) & \text{if } y = 1. \end{cases}$$

The initial guess was $u^{(0)} = 0$. Three different grid sizes, $h = \frac{1}{10}$, $\frac{1}{20}$, and $\frac{1}{40}$, were used. The iterations were continued until the change in u from one iteration to the next did not exceed 10^{-8} . More precisely, the iterations were stopped as soon as

$$\frac{\|u^{(k+1)} - u^{(k)}\|_2}{\|u^{(k+1)}\|_2} < 10^{-8}.$$

The results are shown in Table 7.1. We observe that convergence is slow, and it

h	Matrix dimension	Iterations to convergence
1/10	81	299
1/20	361	1090
1/40	1521	3908

Table 7.1 Jacobi's method applied to model problem

becomes slower as the grid is refined. On the positive side, the program was easy to write and debug. $\square$

Gauss-Seidel Method

Now consider the following simple modification of Jacobi's method. The system to be solved is $Ax = b$, whose i th equation is

$$\sum_{j=1}^n a_{i,j} x_j = b_i.$$

As before, we use the i th equation to modify the i th unknown, but now we consider doing the process sequentially. First we use the first equation to compute $x_1^{(k+1)}$, then we use the second equation to compute $x_2^{(k+1)}$, and so on. By the time we get to the i th equation, we have already computed $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$. In computing $x_i^{(k+1)}$, we could use these newly calculated values, or we could use the old values

$x_1^{(k)}, \dots, x_{i-1}^{(k)}$. The Jacobi method uses the old values; Gauss-Seidel uses the new. That is the only difference. Thus, instead of (7.2.2), Gauss-Seidel performs

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(*)} \right),$$

where $x_j^{(*)}$ denotes the most up-to-date value for the unknown x_j . More precisely, we can write a Gauss-Seidel iteration as follows:

$$\begin{aligned} &\text{for } i = 1, \dots, n \\ &\left[x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right) \right] \end{aligned} \quad (7.2.9)$$

There is no need to maintain separate storage locations for $x_i^{(k)}$ and $x_i^{(k+1)}$; everything is done in a single x array. As soon as $x_\alpha^{(k+1)}$ has been computed, it is stored in place of $x_\alpha^{(k)}$ (which will never be needed again) in location x_α in the array. Thus the iteration (7.2.9) takes the form

$$\begin{aligned} &\text{for } i = 1, \dots, n \\ &\left[x_i \leftarrow \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j \right) \right] \end{aligned} \quad (7.2.10)$$

in practice.

The order in which the corrections are made is important. If they were made in, say, the reverse order $i = n, \dots, 1$, the iteration would have a different outcome. We will always assume that a Gauss-Seidel iteration will be performed in the standard order $i = 1, \dots, n$, as indicated in (7.2.10), unless otherwise stated. The question of orderings is important; more will be said about it in connection with the model problem (7.1.8).

The fact that we can store each new x_i value immediately in place of the old one is an advantage of the Gauss-Seidel method over Jacobi. For one thing, it makes the programming easier. It also saves storage space; Jacobi's method needs to store two copies of x , since $x^{(k)}$ needs to be kept until the computation of $x^{(k+1)}$ is complete. If the system we are solving has millions of unknowns, each copy of x will occupy several megabytes of storage space.

On the other hand, Jacobi's method has the advantage that all of the corrections (7.2.2) can be performed simultaneously; the method is inherently parallel. Gauss-Seidel, in contrast, is inherently sequential, or so it seems. As we shall see, parallelism can be recovered by reordering the equations.

We now investigate the performance of the Gauss-Seidel method by means of some simple examples.

Example 7.2.11 If we apply the Gauss-Seidel method to the same small system as in Example 7.2.3, we obtain, for example,

$$x^{(5)} = \begin{bmatrix} 4.00131 \\ 2.99929 \\ 1.99990 \\ 0.99995 \end{bmatrix} \quad x^{(10)} = \begin{bmatrix} 4.000000142 \\ 2.999999878 \\ 2.000000018 \\ 0.999999993 \end{bmatrix},$$

which are considerably better than the iterates produced by Jacobi's method. Whereas it takes Jacobi's method 50 iterations to obtain an approximation that agrees with the true solution to twelve decimal places, Gauss-Seidel achieves the same accuracy after only 18 iterations. $\square$

Exercise 7.2.12 Given $A \in \mathbb{R}^{n \times n}$, define matrices D , E , and F as follows: $A = D - E - F$, D is diagonal, E is strictly lower triangular, and F is strictly upper triangular. This specifies the matrices completely. For example, $-E$ is the strictly lower triangular part of A .

(a) Show that the Gauss-Seidel iteration (7.2.9) can be written as a matrix equation

$$x^{(k+1)} = D^{-1} \left(b + Ex^{(k+1)} + Fx^{(k)} \right).$$

(b) The vector $x^{(k+1)}$ appears in two places in the equation that you derived in part (a). Solve this equation for $x^{(k+1)}$ to obtain

$$x^{(k+1)} = (D - E)^{-1} \left(b + Fx^{(k)} \right). \quad (7.2.13)$$

(c) Let $M = D - E$. Thus M is the lower triangular part of A , including the main diagonal. Show that $F = M - A$, then verify the formula

$$x^{(k+1)} = x^{(k)} + M^{-1}r^{(k)},$$

where $r^{(k)} = b - Ax^{(k)}$ is the residual. $\square$

Exercise 7.2.14 Write a simple computer program that performs Gauss-Seidel iterations, and use it to verify the results shown in Example 7.2.11. For example, a simple MATLAB code can be built around either of the formulas derived in Exercise 7.2.12, or a Fortran or C code can be built around (7.2.10). $\square$

Exercise 7.2.15 Use the code you wrote for Exercise 7.2.14 to solve the system

$$\begin{bmatrix} 5 & 1 & 1 \\ 1 & 4 & 1 \\ 1 & 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 10 \\ 12 \\ 12 \end{bmatrix}$$

by the Gauss-Seidel method, starting with $x^{(0)} = 0$. Observe the iterates $x^{(k)}$ and the residuals $r^{(k)}$. How many iterations does it take to make $\|r^{(k)}\|_2 < 10^{-5}$? For this value of k , how well does $x^{(k)}$ approximate the true solution? How do your results compare with those of Exercise 7.2.7? $\square$

Now let us see how the Gauss-Seidel method can be applied to the model problem (7.1.8). The equation associated with the (i, j) th grid point is

$$-u_{i-1,j} - u_{i,j-1} + 4u_{i,j} - u_{i,j+1} - u_{i+1,j} = h^2 f_{i,j}.$$

We use it to correct $u_{i,j}$. In the spirit of (7.2.10), we write

$$u_{i,j} \leftarrow \frac{1}{4} [u_{i-1,j} + u_{i,j-1} + u_{i,j+1} + u_{i+1,j} + h^2 f_{i,j}]. \quad (7.2.16)$$

As we have already noted, the outcome of the Gauss-Seidel iteration depends on the order in which the corrections are made. For example, one can sweep through the computational grid by rows (see Figure 7.1), performing the corrections in the order $(1, 1), (2, 1), \dots, (M-1, 1), (1, 2), (2, 2), \dots, (M-1, 2), \dots, (M-1, M-1)$. We will call this the *standard row ordering*. There is an analogous *standard column ordering*. There are also *reverse* row and column orderings, obtained by reversing these. Each ordering gives a slightly different outcome. There are many other orderings that one might consider using.

Example 7.2.17 The Gauss-Seidel method, using the standard row ordering, was applied to the model problem (7.1.8) under exactly the same conditions as in Example 7.2.8. The results are shown in Table 7.2. Comparing with Example 7.2.8,

h	Matrix dimension	Iterations to convergence
1/10	81	160
1/20	361	581
1/40	1521	2082

Table 7.2 Standard Gauss-Seidel applied to model problem

we see that the Gauss-Seidel method needs only slightly more than half as many iterations as the Jacobi method to obtain the same accuracy. It is still fair to say that the convergence is slow, and there is still the problem that the number of iterations increases as the grid is made finer. The largest problem considered here is of modest size. Extrapolating from these results, we can expect Gauss-Seidel will converge very slowly on really big problems. $\square$

Red-Black and Multicolor Gauss-Seidel

Before moving on to a better method, let us spend a little more time on orderings. Red-black orderings are illustrated by Figure 7.3. Grid points are alternately labelled

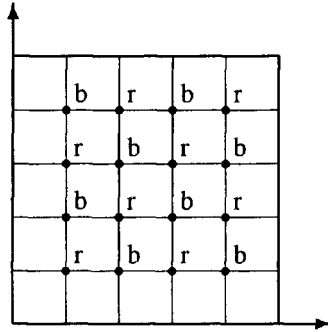


Fig. 7.3 Computational grid of red and black points

red and *black* in a checkerboard pattern. A *red-black ordering* is one in which all of the red points are updated first, then the black points are updated. Notice that every red (resp. black) point has as nearest neighbors only black (resp. red) points (and, in some cases, boundary points). Thus the Gauss-Seidel correction of each red point depends only on the values of u at black points and vice versa. This implies that all of the red points can be updated simultaneously. Once the red points have been done, all of black points can be updated simultaneously. Red-black orderings restore parallelism. Red-black Gauss-Seidel converges at the same rate as standard row-ordered Gauss-Seidel on the model problem (7.1.8). Numerical examples will be given later.

Now consider orderings in the context of the general system $Ax = b$. The standard ordering is $i = 1, \dots, n$. Any other ordering can be obtained by applying the standard ordering to a system $\hat{A}\hat{x} = \hat{b}$ obtained from $Ax = b$ by rearranging the equations (and rearranging the unknowns accordingly). Thus the question of orderings can be translated to a question of permuting the rows and columns of A .

A red-black ordering is one that corresponds to a coefficient matrix with the block structure

$$\hat{A} = \begin{bmatrix} D_1 & B_{12} \\ B_{21} & D_2 \end{bmatrix},$$

where D_1 and D_2 are (square) diagonal matrices of roughly the same size. The first block of rows consists of red equations (with their corresponding red unknowns), and the second block consists of black equations. All of the unknowns of a given color can be updated simultaneously.

Obviously most systems do not admit a red-black ordering. Our model problem (7.1.8) does, and so does its three-dimensional analogue, but many large, sparse problems that arise in applications do not. In these cases a *multicolor* ordering involving more than two colors is often possible. For example, a four-color ordering

corresponds to a coefficient matrix with the block structure

$$\hat{A} = \begin{bmatrix} D_1 & B_{12} & B_{13} & B_{14} \\ B_{21} & D_2 & B_{23} & B_{24} \\ B_{31} & B_{32} & D_3 & B_{34} \\ B_{41} & B_{42} & B_{43} & D_4 \end{bmatrix},$$

where $D_1, \dots, D_4$ are all diagonal matrices of approximately the same size. All of the unknowns within a given block or “color class” can be updated simultaneously.

Symmetric Gauss-Seidel

One iteration of symmetric Gauss-Seidel consists of two standard Gauss-Seidel iterations, one in the forward direction followed by one in the reverse direction. Thus the first half of the iteration is (7.2.10), and the second half is (7.2.10) with “ $i = n, \dots, 1$ ” in place of the standard ordering.

Exercise 7.2.18 In Exercise 7.2.12 you showed that the Gauss-Seidel iteration can be expressed in matrix form as (7.2.13).

- Derive the analogous formula for the reverse Gauss-Seidel method.
- The symmetric Gauss-Seidel iteration consists of a forward Gauss-Seidel step, transforming $x^{(k)}$ to $x^{(k+\frac{1}{2})}$, followed by a reverse Gauss-Seidel step, transforming $x^{(k+\frac{1}{2})}$ to $x^{(k+1)}$. Show that the symmetric Gauss-Seidel iteration satisfies

$$(D - F)x^{(k+1)} = E(D - E)^{-1}Fx^{(k)} + [I + E(D - E)^{-1}]b.$$

- Show that $I + E(D - E)^{-1} = D(D - E)^{-1}$. Then use this fact together with the equation from part (b) to show that the symmetric Gauss-Seidel iteration satisfies

$$Mx^{(k+1)} = Nx^{(k)} + b,$$

where $M = (D - E)D^{-1}(D - F)$, and $N = (D - E)D^{-1}E(D - E)^{-1}F = ED^{-1}F$. Show that M is symmetric if A is.

- Show that the M and N determined in part (c) satisfy $A = M - N$. Then show that the symmetric Gauss-Seidel iteration satisfies

$$x^{(k+1)} = x^{(k)} + M^{-1}r^{(k)}$$

with this choice of M , where $r^{(k)}$ denotes, once again, the residual $b - Ax^{(k)}$.

□

Successive Overrelaxation (SOR)

The process of correcting an equation by modifying one unknown is sometimes called *relaxation*. Before the correction, the equation is not quite true; like an assemblage of parts that does not fit together quite right, it is in a state of tension. The correction of one variable relaxes the tension. The Gauss-Seidel method performs *successive relaxation*. That is, it moves from equation to equation, relaxing one after the other.²

In many cases convergence can be accelerated substantially by overrelaxing. This means that rather than making a correction for which the equation is satisfied exactly, we make a somewhat bigger correction. In the simplest case one chooses a *relaxation factor* $\omega > 1$ and overcorrects by that factor at each step. This is *successive overrelaxation* (SOR):

$$\begin{array}{l} \text{for } i = 1, \dots, n \\ \left[\begin{array}{l} \hat{x} \leftarrow \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j \right) \\ \delta \leftarrow \hat{x} - x_i \\ x_i \leftarrow x_i + \omega \delta \end{array} \right. \end{array} \quad (7.2.19)$$

Comparing (7.2.19) with (7.2.10), we see that SOR collapses to Gauss-Seidel when $\omega = 1$. One could also use a factor ω that is less than one, which amounts to underrelaxation, but this normally slows convergence.

Since SOR is “successive” (like Gauss-Seidel) rather than “simultaneous” (like Jacobi), it needs to keep only one copy of the vector x .

Not much is gained by applying overrelaxation to the small problem featured in Examples 7.2.3 and 7.2.11, so let us move directly to the model problem (7.1.8). Referring to the Gauss-Seidel step (7.2.16), we see that the corresponding SOR step is

$$\begin{array}{l} \hat{u} \leftarrow \frac{1}{4} [u_{i-1,j} + u_{i,j-1} + u_{i,j+1} + u_{i+1,j} + h^2 f_{i,j}] \\ \delta \leftarrow \hat{u} - u_{i,j} \\ u_{i,j} \leftarrow u_{i,j} + \omega \delta. \end{array} \quad (7.2.20)$$

Just as for Gauss-Seidel, the outcome of an SOR iteration depends on the order in which the updates are made. Commonly used orderings include the standard row and column orderings and the red-black ordering illustrated by (7.3). One can also do *symmetric* SOR (abbreviated SSOR), which consists of alternating forward and backward standard row (or column) sweeps.

Example 7.2.21 The SOR method, using the standard row ordering, was applied to the model problem (7.1.8) under exactly the same conditions as in Examples 7.2.8 and 7.2.17. Several values of the relaxation factor ω were tried. The results are

²In contrast, Jacobi’s method performs *simultaneous relaxation*.

shown in Table 7.3. Iterations were halted as soon as

$$\frac{\|u^{(k+1)} - u^{(k)}\|_2}{\|u^{(k+1)}\|_2} < 10^{-8}.$$

The choice $\omega = 1$ corresponds to Gauss-Seidel; the numbers in the second row are

	$h = 1/10$	$1/20$	$1/40$
$\omega = 0.8$	235	845	3018
1.0	160	581	2082
1.4	67	262	955
1.6	42	151	577
1.7	57	96	412
1.8	86	89	252
1.9	176	180	179
2.0	∞	∞	∞

Table 7.3 Iterations counts for SOR applied to model problem

identical to those in Table 7.2. We note that underrelaxation ($\omega = 0.8$) is detrimental and overrelaxation can be very beneficial, particularly if an ω that is close to optimal is used. For the finest mesh, the iteration count can be cut by a factor of ten or more. Since SOR steps take only slightly more work than Gauss-Seidel steps, the savings in iterations translate almost directly into savings in computer time.

From the numbers in Table 7.3 we see that the optimal value of ω depends on the mesh size (and in general it depends on the coefficient matrix A). For $h = 1/10$ the best ω is somewhere around 1.6, for $h = 1/20$ it is around 1.8, and for $h = 1/40$ it is around 1.9. For most problems the optimal ω is not known, but the numbers in Table 7.3 suggest that even a crude guess can yield substantial savings. The numbers also suggest that it is better to overestimate than to underestimate the optimum. However, SOR diverges when $\omega \geq 2$.

For our model problem the optimal ω is known to be

$$\omega_b = \frac{2}{1 + \sin \pi h}.$$

When $h = 1/40$ it is approximately 1.855. Using this value of ω , SOR converges in 137 iterations. $\square$

As an exercise, you can explore the convergence of the red-black SOR and SSOR methods (Exercise 7.2.23). On the model problem (and many similar ones) red-black and standard row ordered SOR converge at the same rate. SSOR is somewhat slower but less sensitive to the choice of the relaxation parameter ω .

The computations reported in Example 7.2.21 were performed by a simple Fortran program based on (7.2.20). An iteration with the standard row ordering is effected

by doubly nested loops:

$$\begin{array}{l} \text{for } j = 1, \dots, m-1 \\ \quad \left[\begin{array}{l} \text{for } i = 1, \dots, m-1 \\ \quad \left[\begin{array}{l} \hat{u} \leftarrow \frac{1}{4} (u_{i-1,j} + u_{i,j-1} + u_{i,j+1} + u_{i+1,j} + h^2 f_{i,j}) \\ \delta \leftarrow \hat{u} - u_{i,j} \\ u_{i,j} \leftarrow u_{i,j} + \omega \delta. \end{array} \right. \end{array} \right. \end{array}$$

The implementations of red-black SOR and SSOR are only slightly more complicated. In particular, it was not necessary to form or manipulate the coefficient matrix A or any related matrix.

In Exercises 7.2.4 and 7.2.12 you derived the matrix formulas (7.2.5) and (7.2.13) for the Jacobi and Gauss-Seidel iterations, respectively. A similar expression can be derived for the SOR method (Exercise 7.2.24). These formulas are useful for analyzing the convergence of the methods, as will be illustrated in Section 7.3. They are also handy for writing simple MATLAB programs to apply to small examples. However, it is important to realize that these formulas should not be used for large problems, as the resulting code would be horribly inefficient. Instead one should write code as described in the previous paragraph.

Block Iterative Methods

All of the iterative methods that we have derived in this section have block variants. Suppose the system $Ax = b$ has been subdivided into blocks:

$$A = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1q} \\ A_{21} & A_{22} & \cdots & A_{2q} \\ \vdots & \vdots & \ddots & \vdots \\ A_{q1} & A_{q2} & \cdots & A_{qq} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_q \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_q \end{bmatrix},$$

where the main diagonal blocks A_{ii} are all square and nonsingular. The entries x_i and b_i now refer to subvectors of x and b of the appropriate lengths. A block Jacobi iteration is just like an ordinary Jacobi iteration, except that it acts on blocks instead of the matrix entries. Instead of (7.2.2), we have

$$x_i^{(k+1)} = A_{ii}^{-1} \left(b_i - \sum_{j \neq i} A_{ij} x_j^{(k)} \right). \quad (7.2.22)$$

Block Gauss-Seidel and SOR iterations are derived in similar fashion. In order that such a block algorithm be efficient, it is necessary that the main diagonal blocks A_{ii} be simple enough that their inverses (or LU decompositions) can be obtained easily.

Consider, for example, the matrix of the model problem (7.1.8), written in the block form (7.1.10). To apply a block algorithm using this blocking, we need to be able to LU decompose the tridiagonal matrix T . As we know, this is simple. Each block corresponds to one row (or *line*) of unknowns in the computational grid, so

block algorithms based on this partition are known as *line* iterations, for example, line Gauss-Seidel or line SOR.

Red-Black (Gauss-Seidel or SOR) iterations can be viewed as block iterations based on a 2×2 blocking

$$\begin{bmatrix} D_1 & A_{12} \\ A_{21} & D_2 \end{bmatrix}.$$

More generally, a multicolor iteration with q colors is a block iteration based on a $q \times q$ blocking. Here the matrices on the main diagonal are trivial to invert, as they are diagonal matrices.

Additional Exercises

Exercise 7.2.23 Write programs that solve the model problem (7.1.8) by standard SOR, red-black SOR, and SSOR. Use each of them to solve the model problem with $f = 0$ and g given by

$$g(x, y) = \begin{cases} 0 & \text{if } x = 0 \\ y & \text{if } x = 1 \\ (x - 1) \sin x & \text{if } y = 0 \\ x(2 - x) & \text{if } y = 1, \end{cases}$$

as in Example 7.2.8. Use mesh size $h = 1/40$. Starting from $u^{(0)} = 0$, record how many iterations are needed until

$$\frac{\|u^{(k+1)} - u^{(k)}\|_2}{\|u^{(k+1)}\|_2} < 10^{-8}.$$

Since SSOR iterations take twice as much work as the other iterations, count each SSOR iteration as two iterations. For each of the three methods try $\omega = 1.0, 1.1, 1.2, \dots, 1.9$. Compare the performance of the three methods. $\square$

Exercise 7.2.24 (Matrix representation of SOR)

- (a) In the SOR iteration (7.2.19), show that successive iterates $x^{(k)}$ and $x^{(k+1)}$ are related by

$$a_{ii}x_i^{(k+1)} = a_{ii}x_i^{(k)} + \omega \left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i}^n a_{ij}x_j^{(k)} \right].$$

- (b) Write $A = D - E - F$, as in Exercise 7.2.12. Derive the following expressions for the relationship between successive SOR iterates $x^{(k)}$ and $x^{(k+1)}$.

$$Dx^{(k+1)} = Dx^{(k)} + \omega [b + Ex^{(k+1)} + (F - D)x^{(k)}].$$

$$\left(\frac{1}{\omega} D - E \right) x^{(k+1)} = \left(\frac{1 - \omega}{\omega} D + F \right) x^{(k)} + b. \quad (7.2.25)$$

Thus $Mx^{(k+1)} = Nx^{(k)} + b$, where $M = \frac{1}{\omega}D - E$, $N = \frac{1-\omega}{\omega}D + F$, and $A = M - N$.

(c) Show that

$$x^{(k+1)} = x^{(k)} + M^{-1}r^{(k)},$$

where $r^{(k)} = b - Ax^{(k)}$.

□

Exercise 7.2.26 (Matrix representation of SSOR) This exercise generalizes Exercise 7.2.18

- (a) Find the formula analogous to (7.2.25) for an SOR iteration in the reverse direction.
- (b) The symmetric SOR (SSOR) iteration consists of a forward SOR step, transforming $x^{(k)}$ to $x^{(k+\frac{1}{2})}$, followed by a reverse SOR step, transforming $x^{(k+\frac{1}{2})}$ to $x^{(k+1)}$. Show that the SSOR iteration satisfies

$$\begin{aligned} \left(\frac{1}{\omega}D - F\right)x^{(k+1)} &= \left(\frac{1-\omega}{\omega}D + E\right)\left(\frac{1}{\omega}D - E\right)^{-1}\left(\frac{1-\omega}{\omega}D + F\right)x^{(k)} \\ &\quad + \left[I + \left(\frac{1-\omega}{\omega}D + E\right)\left(\frac{1}{\omega}D - E\right)^{-1}\right]b. \end{aligned}$$

- (c) Show that $I + \left(\frac{1-\omega}{\omega}D + E\right)\left(\frac{1}{\omega}D - E\right)^{-1} = \frac{2-\omega}{\omega}D\left(\frac{1}{\omega}D - E\right)^{-1}$. Then use this fact together with the equation from part (b) to show that the SSOR iteration satisfies

$$Mx^{(k+1)} = Nx^{(k)} + b,$$

where

$$M = \frac{\omega}{2-\omega}\left(\frac{1}{\omega}D - E\right)D^{-1}\left(\frac{1}{\omega}D - F\right),$$

and

$$N = \frac{\omega}{2-\omega}\left(\frac{1-\omega}{\omega}D + E\right)D^{-1}\left(\frac{1-\omega}{\omega}D + F\right).$$

Show that M is symmetric if A is.

- (d) Show that the M and N determined in part (c) satisfy $A = M - N$. Then show that the SSOR iteration satisfies

$$x^{(k+1)} = x^{(k)} + M^{-1}r^{(k)}$$

with this choice of M , where $r^{(k)} = b - Ax^{(k)}$.

□

Exercise 7.2.27 (Smoothing property of Gauss-Seidel iterations) Consider the linear system $Ax = b$, where A is the 10×10 version of the tridiagonal matrix (7.1.5) and

$$b = \begin{bmatrix} 2 & -2 & 2 & -1 & 0 & 0 & 1 & -2 & 2 & -2 \end{bmatrix}^T.$$

You can easily check that the exact solution to this system is

$$x = [1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ -1 \ 0 \ -1]^T.$$

Starting with $x^{(0)} = 0$, perform at least four Gauss-Seidel iterations. Since the exact solution of the system is known, you can calculate the errors $e^{(i)} = x - x^{(i)}$. Make plots of the errors; that is, given an error vector e , plot the points (i, e_i) and connect them with line segments ("plot(e)" or "plot(1:10,e)" in MATLAB). Notice that the plot of $e^{(0)}$ is quite jagged, but each successive plot is less so. The plot of $e^{(4)}$ is quite smooth.

Multigrid methods are fast iterative methods that solve problems like (7.1.8) rapidly by applying smoothing operations and coarse-grid corrections recursively. Gauss-Seidel is a popular smoothing operator for multigrid algorithms. $\square$

Exercise 7.2.28

- (a) Repeat Exercise 7.2.27 using Jacobi iterations in place of Gauss-Seidel. Notice that the Jacobi method does not smooth the error. Thus the basic Jacobi method is useless as a multigrid smoothing operator.
- (b) The *damped Jacobi method* with *damping factor* $\omega < 1$ is the same as the ordinary Jacobi method, except that the correction at each step is damped by the factor ω . Thus, instead of $x^{(k+1)} = x^{(k)} + D^{-1}r^{(k)}$ (cf. Exercise 7.2.4), damped Jacobi makes the correction

$$x^{(k+1)} = x^{(k)} + \omega D^{-1}r^{(k)}.$$

Repeat Exercise 7.2.27 using the Damped Jacobi method with $\omega = 1/2$ and $\omega = 2/3$. The damped Jacobi iteration with $1/2 \leq \omega \leq 2/3$ is an effective smoother.

$\square$

7.3 CONVERGENCE OF ITERATIVE METHODS

Splittings

We unify the convergence theory of iterative methods by introducing the notion of a splitting. Given an $n \times n$ nonsingular matrix A , a *splitting* of A is nothing but an additive decomposition $A = M - N$, where M is nonsingular. The matrix M is called the *splitting matrix*. As we shall later see, it can also be called a *preconditioner*.

Every splitting gives rise to an iterative method as follows: Use the splitting to rewrite the system $Ax = b$ in the form $Mx = Nx + b$ or $x = M^{-1}Nx + M^{-1}b$. Then define an iteration by either of the equivalent equations

$$x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b$$

or

$$Mx^{(k+1)} = Nx^{(k)} + b. \quad (7.3.1)$$

Clearly this scheme requires that M be nonsingular. From a practical standpoint it is also important that M be “easy to solve,” meaning that systems with M as the coefficient matrix are easy to solve. If the iterates converge to some x , then that x satisfies $Mx = Nx + b$, hence $Ax = b$. For the sake of rapid convergence, we would like $M \approx A$ and $N \approx 0$. From this viewpoint the best choice is $M = A$, which gives convergence in one iteration, but this M violates the “easy to solve” requirement. What is needed is a good compromise: M should approximate A as well as possible without being too complicated.

All of the iterative methods that we have discussed so far can be induced by splittings of A . Table 7.4 lists the splitting matrices M associated with the classical iterations, as determined in Exercises 7.2.4, 7.2.12, 7.2.24, and 7.2.26. The matrices D , E , and F refer to the decomposition $A = D - E - F$, where D , $-E$, and $-F$ are the diagonal, lower triangular, and upper triangular parts of A , respectively. In

Method	Splitting Matrix (M)	Exercise
Jacobi	D	7.2.4
Gauss-Seidel	$D - E$	7.2.12
SOR	$\frac{1}{\omega}D - E$	7.2.24
SSOR	$\frac{\omega}{2-\omega} \left(\frac{1}{\omega}D - E \right) D^{-1} \left(\frac{1}{\omega}D - F \right)$	7.2.26

Table 7.4 Splitting matrices for classical iterations

each case only M has been listed, because N can be inferred from the relationship $N = M - A$.

Example 7.3.2 A very simple method that plays a surprisingly big role in convergence theory is *Richardson’s method*, which is induced by the splitting $M = \frac{1}{\omega}I$, $N = \frac{1}{\omega}I - A$, where ω is a *damping factor* chosen to make M approximate A as well as possible. Thus Richardson’s iteration is

$$x^{(k+1)} = (I - \omega A)x^{(k)} + \omega b.$$

The properties of Richardson’s method are explored in Exercises 7.3.19, 7.3.20, 7.3.22, and 7.3.37. $\square$

Exercise 7.3.3 Suppose we apply Richardson’s method to the model problem (7.1.8). What would be the obvious choice of ω to try, given that we want M to approximate A as well as possible? For this choice of ω , with which classical iteration does Richardson’s method coincide? $\square$

Convergence

We consider an iteration of the form (7.3.1). At each step there is an *error* $e^{(k)} = x - x^{(k)}$, which is just the difference between the true solution and its approximant after k iterations. As we are in the process of solving a problem, we will not know what $e^{(k)}$ is (because x is not yet known), but that does not stop us from talking about it. For a given splitting $A = M - N$, we would like to prove that $e^{(k)} \rightarrow 0$ as $k \rightarrow \infty$. Moreover, since we cannot wait forever in practice, we would like to show, if possible, that the $e^{(k)}$ become small quickly. The true solution of $Ax = b$ also satisfies $Mx = Nx + b$. Subtracting the equation (7.3.1) from this, we find that $Me^{(k+1)} = Ne^{(k)}$. Thus

$$e^{(k+1)} = Ge^{(k)},$$

where $G = M^{-1}N = I - M^{-1}A$. As this equation holds for all k , we have $e^{(1)} = Ge^{(0)}$, $e^{(2)} = Ge^{(1)} = G^2e^{(0)}$, and, in general,

$$e^{(k)} = G^k e^{(0)}. \quad (7.3.4)$$

The vector $e^{(0)}$ is our initial error; its size depends on the initial guess. Equation (7.3.4) shows that regardless of what the initial guess was, $e^{(k)} \rightarrow 0$ if $G^k \rightarrow 0$. Since powers of a matrix are what matters, it should not be surprising that the convergence theory of iterative methods resembles the analysis of the power method for computing a dominant eigenvector. In particular, the eigenvalues of the iteration matrix play a crucial role. But now the scenario is somewhat different; we want the iterates to converge to zero, not to an eigenvector.

Let us assume that the iteration matrix G is semisimple. (For the non-semisimple case, see Exercise 7.3.27.) This means that G has n linearly independent eigenvectors $v_1, v_2, \dots, v_n$, which form a basis of $\mathbb{C}^n$. Let $\lambda_1, \lambda_2, \dots, \lambda_n$ denote the corresponding eigenvalues of G : $Gv_i = \lambda_i v_i$, $i = 1, \dots, n$. We will focus on the errors $e^{(k)}$ directly rather than the matrices G^k (but see Exercise 7.3.26). Since the eigenvectors $v_1, \dots, v_n$ form a basis of $\mathbb{C}^n$, we can express the initial error $e^{(0)}$ as a linear combination of them:

$$e^{(0)} = c_1 v_1 + c_2 v_2 + \dots + c_n v_n. \quad (7.3.5)$$

The scalars $c_1, \dots, c_n$ are unknown, since $e^{(0)}$ is, and they might be complex numbers, even though $e^{(0)}$ is real. None of this causes any problems for the analysis. Applying G to (7.3.5), we find that

$$e^{(1)} = Ge^{(0)} = c_1 \lambda_1 v_1 + c_2 \lambda_2 v_2 + \dots + c_n \lambda_n v_n,$$

and in general

$$e^{(k)} = G^k e^{(0)} = c_1 \lambda_1^k v_1 + c_2 \lambda_2^k v_2 + \dots + c_n \lambda_n^k v_n.$$

Consequently

$$\|e^{(k)}\| \leq |c_1| |\lambda_1|^k \|v_1\| + |c_2| |\lambda_2|^k \|v_2\| + \dots + |c_n| |\lambda_n|^k \|v_n\|,$$

where the norm can be any vector norm. Since $|\lambda_i|^k \rightarrow 0$ if and only if $|\lambda_i| < 1$, we conclude that $\|e^{(k)}\| \rightarrow 0$ for every initial guess $x^{(0)}$ if and only if $\max_i |\lambda_i| < 1$.

Let $\sigma(G)$ denote the spectrum of G , the set of eigenvalues. The *spectral radius*, denoted $\rho(G)$, is the maximum distance of an eigenvalue from the origin. Thus

$$\rho(G) = \max_{\lambda \in \sigma(G)} |\lambda|.$$

Using this new terminology we can restate our conclusion from the previous paragraph: The iterations converge (for any starting vector) if and only if $\rho(G) < 1$.

The spectral radius also gives information about the rate of convergence. If, say, λ_1 is the eigenvalue of greatest modulus ($|\lambda_1| = \rho(G)$), we eventually have

$$e^{(k)} \approx c_1 \lambda_1^k v_1,$$

since the other λ_i^k tend to zero more quickly than λ_1^k does, so

$$\frac{\|e^{(k+1)}\|}{\|e^{(k)}\|} \approx |\lambda_1| = \rho(G) \quad (7.3.6)$$

for sufficiently large k . Thus the convergence is linear with convergence ratio $\rho(G)$. The smaller $\rho(G)$ is, the faster the iterations converge.

This argument depends on the assumption $c_1 \neq 0$, which holds for almost all choices of $x^{(0)}$. For those (exceedingly rare) vectors that yield $c_1 = 0$, the convergence is typically faster in theory. (Why?)

The approximation (7.3.6) is certainly valid for large k . It does not necessarily hold for small k .

We have now nearly (cf. Exercise 7.3.27) proved the following Theorem.

Theorem 7.3.7 *The iteration (7.3.1) converges to the true solution of $Ax = b$ for every initial guess $x^{(0)}$ if and only if the spectral radius of the iteration matrix $G = I - M^{-1}A$ is less than one. The convergence is linear. The average convergence ratio never exceeds $\rho(G)$; it equals $\rho(G)$ for almost all choices of $x^{(0)}$.*

In general it is difficult to calculate $\rho(G)$, but $\rho(G)$ is known in some special cases. In particular, for the model problem (7.1.8), $\rho(G)$ is known for the Richardson, Jacobi, Gauss-Seidel, and SOR (standard and red-black) iterations. There is not room here to derive all of these results; fortunately there are some excellent references, for example, [36], [38], [76], [84]. We will just scratch the surface.

The Jacobi iteration matrix is $G = D^{-1}(E + F)$. For the model problem (7.1.8), we have $G = \frac{1}{4}(E + E^T)$. In Exercise 7.3.35 it is shown that in this case $\rho(G) = \cos(\pi h)$, where h , as always, is the mesh size. It is not surprising that $\rho(G)$ would depend on h , particularly in light of the results of Example 7.2.8. We have $\rho(G) < 1$, so Jacobi's method is convergent for any h . However, as $h \rightarrow 0$, $\cos(\pi h) \rightarrow 1$, which implies that the convergence becomes very slow as h is made small. To get a more quantitative view of the situation, recall the Taylor expansion $\cos z = 1 - z^2/2! + O(z^4)$, which implies

$$\rho(G) \approx 1 - \frac{\pi^2}{2} h^2. \quad (7.3.8)$$

Each time h is cut by a factor of 2, the distance between $\rho(G)$ and 1 is cut by a factor of about 4. For example, when $h = 1/10, 1/20$, and $1/40$, we have $\rho(G) = .9511, .9877$, and $.9969$, respectively.

It is clear that with a spectral radius like $.9969$, convergence will be very slow. Let us estimate how many iterations are needed to reduce the error by a factor of 10. By (7.3.6) we have

$$\frac{\|e_{k+j}\|}{\|e_k\|} \approx \rho(G)^j.$$

Thus, to shrink the error by a factor of 10, we will need about j iterations, where $\rho(G)^j \approx 10^{-1}$. Applying logarithms and solving for j , we find that

$$j \approx \frac{\log 10}{-\log \rho(G)}.$$

For example, for $\rho = .9969$, we get $j \approx 742$. That is, about 742 iterations are needed for each reduction of the error by a factor of 10. If we wish to reduce the error by a factor of, say, 10^{-8} , we will need about $8 \times 742 \approx 5930$ iterations.

In the computation of j , any base of logarithms can be used. Base 10 looks inviting, but for theoretical work, base e is preferred. The *asymptotic convergence rate* of an iterative method, denoted $R_\infty(G)$, is defined by

$$R_\infty(G) = -\log_e \rho(G). \quad (7.3.9)$$

The subscript ∞ is meant to convey the idea that this number is useful when the number of iterations is large (i.e. as $k \rightarrow \infty$) and not necessarily for small k .

Exercise 7.3.10 Show that $1/R_\infty(G)$ is approximately the number of iterations needed to reduce the error by a factor of e (once k is sufficiently large). $\square$

When $\rho(G) = 1$ we have $R_\infty(G) = 0$, indicating no convergence. When $\rho(G) < 1$, we have $R_\infty(G) > 0$. The smaller $\rho(G)$ is, the larger $R_\infty(G)$ will be. As $\rho(G) \rightarrow 0$, we have $R_\infty(G) \rightarrow \infty$.

A doubling of $R_\infty(G)$ implies a doubling of the convergence rate, in the sense that only about half as many iterations will be required to reduce the error by a given factor.

When Jacobi's method is applied to the model problem, the spectral radius is given by (7.3.8). Using this and the Taylor expansion $-\log(1-z) = z + \frac{1}{2}z^2 + \frac{1}{3}z^3 + \cdots$, we find that

$$R_\infty(G) = \frac{\pi^2}{2}h^2 + O(h^4).$$

Stating this result more succinctly, we have $R_\infty(G) = O(h^2)$. This means that each time we cut h in half, we cut $R_\infty(G)$ by a factor of 4, which means, in turn, that about four times as many iterations will be needed to attain a given accuracy. This theoretical conclusion is borne out by the numbers in Example 7.2.8.

Exercise 7.3.11 Calculate $\rho(G)$ and $R_\infty(G)$ for Jacobi's method applied to the model problem (7.1.8) with $h = .02$ and $h = .01$. For each of these choices of h , about how many iterations are required to reduce the error by a factor of 10^{-3} ? $\square$

The numbers in Example 7.2.17 suggest that the Gauss-Seidel method converges somewhat faster than the Jacobi method does. Indeed this is true for the model problem and any problem that admits a red-black ordering. Let G_J and G_{GS} denote the iteration matrices for Jacobi and red-black Gauss-Seidel, respectively. In Exercise 7.3.38 you will show that $\rho(G_{GS}) = \rho(G_J)^2$. The standard Gauss-Seidel iteration has the same spectral radius as red-black Gauss-Seidel, because the standard ordering is a so-called *consistent ordering* [38], [76], [84]. This means that one Gauss-Seidel iteration is about as good as two Jacobi iterations; Gauss-Seidel converges twice as fast as Jacobi. The numbers in Examples 7.2.8 and 7.2.17 bear this out.

Exercise 7.3.12 Show that if $\rho(G_{GS}) = \rho(G_J)^2$, then $R_\infty(G_{GS}) = 2R_\infty(G_J)$. □

For the model problem (7.1.8) we have $\rho(G_{GS}) = \rho(G_J)^2 = \cos^2 \pi h$ and $R_\infty(G_{GS}) = \pi^2 h^2 + O(h^4)$. Gauss-Seidel is just as bad as Jacobi in the sense that $R_\infty(G_{GS}) = O(h^2)$. Each time h is halved, the number of iterations needed is multiplied by four. Clearly something better is needed.

Example 7.2.21 indicates that the SOR method with a good relaxation factor ω is substantially better than Gauss-Seidel. Indeed it can be shown that SOR with the optimal ω satisfies $R_\infty(G) \approx 2\pi h$. This and other results are summarized in

Method	$\rho(G)$	$R_\infty(G)$
Jacobi	$\cos \pi h$	$\frac{1}{2}\pi^2 h^2 + O(h^4)$
Gauss-Seidel	$\cos^2 \pi h$	$\pi^2 h^2 + O(h^4)$
SOR (optimal ω)	$1 - 2\pi h + O(h^2)$	$2\pi h + O(h^2)$
SSOR (good ω)	$\leq 1 - \pi h + O(h^2)$	$\geq \pi h + O(h^2)$

Table 7.5 Convergence rates of classical iterations on model problem (7.1.8)

Table 7.5. Since $R_\infty(G) = O(h)$, each halving of h only doubles the number of iterations required for SOR to achieve a given reduction in error. Thus SOR promises to perform significantly better than Gauss-Seidel when h is small.

Exercise 7.3.13 For $h = .02$ and $h = .01$, estimate $R_\infty(G)$ for SOR with optimal ω on the model problem (7.1.8). For both of these h , about how many iterations are required to reduce the error by a factor of 10^{-3} ? Compare your results with those of Exercise 7.3.11 □

Although SOR is a huge improvement over Jacobi and Gauss-Seidel, it still is not optimal. Ideally one would like an iterative method for which $R_\infty(G)$ is independent of h . Then the number of iterations required to reach a given accuracy would also be independent of h . If the cost of an iteration is $O(n)$, that is, proportional to the number of equations, then the cost of solving the system to a given accuracy will also be $O(n)$, since the number of iterations is independent of n . Multigrid and domain decomposition techniques [9], [36], [63] achieve this ideal for the model

problem (7.1.8) and a wide variety of other systems obtained from discretization of partial differential equations. A discussion of these important techniques is beyond the scope of this book.

The convergence theory of the classical iterative methods is laid out in [36], [76], and [84], for example. For the model Poisson problem the theory is complete, and most of the theory carries over to other positive definite systems that admit a red-black ordering: The convergence rates of all of the classical iterative methods are determined by the spectral radius of the Jacobi iteration matrix.

So far we have restricted our attention to simple iterative methods with no memory. These methods construct $x^{(k+1)}$ from $x^{(k)}$; they make no use of $x^{(k-1)}$ or earlier iterates. An advantage of these methods is that they economize on memory: Old iterates need not be saved. One might wonder, nevertheless, whether there might possibly be some gain from saving a few previous iterates and somehow making use of the information that they contain. They would give an idea of the general trend of the iterations and might be used to extrapolate to a much better estimate of the solution. This train of ideas was pursued successfully by Golub and Varga, who invented the Chebyshev semi-iterative method, also known as Chebyshev acceleration. A related technique is conjugate gradient acceleration. Both of these acceleration techniques generate $x^{(k+1)}$ from just $x^{(k)}$ and $x^{(k-1)}$. A good reference is [38].

Instead of developing acceleration techniques, we will take a different approach. The conjugate gradient algorithm will be derived as an example of a descent method. We will see that the convergence of descent algorithms can be enhanced significantly by use of preconditioners. It turns out that applying the conjugate gradient method with preconditioner M (as described later in this chapter) is equivalent to applying conjugate gradient acceleration (as described, e.g. in [38]) to a basic iteration (7.3.1) with splitting matrix M .

Additional Exercises

Exercise 7.3.14 This and the next few exercises explore the computational costs of the Gauss-Seidel and SOR methods. The coefficient matrix of the model problem (7.1.8) with $h = 1/m$ has $n \approx m^2$ equations in as many unknowns. We cannot hope to solve $Au = b$ in fewer than $O(m^2)$ flops. In Section 7.1 we observed that banded Gaussian elimination requires $\frac{1}{2}m^4$ flops.

- Show that the number of flops required to do one iteration of Gauss-Seidel or SOR is a small multiple of m^2 .
- Show that the number of Gauss-Seidel iterations needed to decrease the error by a fixed factor ϵ is approximately Cm^2 , where $C = -(\log_e \epsilon)/\pi^2$.
- Estimate the overall flop count for solving $Au = b$ by Gauss-Seidel. How does this compare to the flop count for banded Gaussian elimination?
- Estimate the number of SOR iterations (with optimal ω) needed to decrease the error by a fixed factor ϵ .

- (e) Show that the overall flop count for SOR with optimal ω is much less than for banded Gaussian elimination, if h is sufficiently small.
- (f) How does SOR compare with a method that takes $O(m^2)$ flops per iteration and converges in some number of iterations that is independent of h ?

□

Exercise 7.3.15 Repeat Exercise 7.3.14 for the three-dimensional model problem (see (7.1.11) and Exercises 7.1.12 and 7.1.13), bearing in mind that some of the conclusions may be different. In particular, how do the iterative methods compare to banded Gaussian elimination? (The spectral radii for the three-dimensional problem are the same as for the two-dimensional problem.) □

Exercise 7.3.16 The solution of the model problem (7.1.8) is an approximation to the solution of a boundary value problem. The smaller h is, the better the approximation will be, and the more worthwhile it will be to solve $Au = b$ accurately. A more careful flop count will take this into account. If the exact solution of the boundary value problem is smooth enough, the solution of (7.1.8) will differ from it by approximately Ch^2 , where C is a constant. It therefore makes sense to solve $Au = b$ with accuracy ch^2 , where $c < C$.

- (a) Rework parts (b) through (e) of Exercise 7.3.14 under the assumption that we need to do enough iterations to reduce the error by a factor ϵh^2 . Show that this just changes the results by a factor $K \log m$ (when m is large).
- (b) Calculate $\log m$ for $m = 100, 10000, 1000000$. This is meant to remind you of how slowly the logarithm grows.
- (c) Is the exponent of h important? Suppose we need to reduce the error by a factor ϵh^s , where s is some number other than 2. Does this change the general conclusion of the analysis?

□

Exercise 7.3.17 Consider solving the model problem (7.1.8) with $f = 0$ and g given by

$$g(x, y) = \begin{cases} 0 & \text{if } x = 0 \\ y & \text{if } x = 1 \\ (x - 1) \sin x & \text{if } y = 0 \\ x(2 - x) & \text{if } y = 1, \end{cases}$$

as in Example 7.2.8, using mesh size $h = 1/10$.

- (a) Referring to Table 7.5, calculate $\rho(G)$ and $R_\infty(G)$ for Jacobi's method applied to this problem. Based on these quantities, about how many iterations should it take to reduce the error by a factor of 100?

- (b) Write a computer program that tests the theory. A crude MATLAB code will be good enough. Have the program begin by computing an accurate solution (error $< 10^{-10}$) of $Au = b$ by SOR or any other method. Use this as an “exact solution” against which to compare your Jacobi iterates. Do Jacobi iterates, starting with $u^{(0)} = 0$. At each step calculate the norm of the error: $\|e^{(k+1)}\|_2 = \|u - u^{(k+1)}\|_2$ and the ratio $\|e^{(k+1)}\|_2 / \|e^{(k)}\|_2$. Observe the ratios for 100 steps or so. Are they close to what you would expect?
- (c) Observe how many iterations it takes to reduce $\|e^{(k)}\|_2$ from 10^{-1} to 10^{-3} . Does this agree well with theory?
- (d) The first iterations make more progress toward convergence than subsequent iterations do. How might this effect be explained?

□

Exercise 7.3.18 Repeat the previous exercise using Gauss-Seidel (standard row ordering) instead of Jacobi. □

Exercise 7.3.19 (Convergence of Richardson’s method) Consider a linear system $Ax = b$, where the eigenvalues of A are real and positive. For example, A could be positive definite. Let λ_1 and λ_n denote the smallest and largest eigenvalues, respectively, of A . Let $G_\omega = I - \omega A$ denote the iteration matrix of Richardson’s method applied to A .

- (a) Show that all of the eigenvalues of G_ω are less than one.
- (b) Prove that Richardson’s method converges if and only if $\omega < 2/\lambda_n$.
- (c) Prove that the optimal value of ω (minimizing $\rho(G_\omega)$) is $\omega_b = \frac{2}{\lambda_n + \lambda_1}$. More precisely, show that

$$\rho(G_\omega) = \begin{cases} 1 - \omega\lambda_1 & \text{if } \omega \leq \omega_b \\ \frac{\lambda_n - \lambda_1}{\lambda_n + \lambda_1} & \text{if } \omega = \omega_b \\ \omega\lambda_n - 1 & \text{if } \omega \geq \omega_b. \end{cases}$$

- (d) Show that if A is symmetric and positive definite, then

$$\rho(G_{\omega_b}) = \frac{\kappa_2(A) - 1}{\kappa_2(A) + 1},$$

where $\kappa_2(A)$ denotes the spectral condition number of A (cf. Theorem 4.2.4 and Exercise 5.4.46). Thus (optimal) Richardson’s method converges rapidly when A is well conditioned and slowly when A is ill conditioned.

- (e) Show that the convergence rate of (optimal) Richardson’s method is

$$R_\infty(G_{\omega_b}) = -\log \left(\frac{1 - \kappa^{-1}}{1 + \kappa^{-1}} \right),$$

where $\kappa^{-1} = 1/\kappa_2(A)$. Use Taylor expansions to show that $R_\infty(G_{\omega_b}) = 2\kappa^{-1} + O(\kappa^{-2})$.

□

Exercise 7.3.20 Show that any iteration that is generated by a splitting $A = M - N$ can be expressed as a correction based on the residual $r^{(k)} = b - Ax^{(k)}$:

$$x^{(k+1)} = x^{(k)} + M^{-1}r^{(k)}. \quad (7.3.21)$$

In particular, Richardson's iteration can be written as $x^{(k+1)} = x^{(k)} + \omega r^{(k)}$. □

Exercise 7.3.22 (Preconditioners) If we wish to solve a system $Ax = b$, we can equally well solve an equivalent system $M^{-1}Ax = M^{-1}b$, where M is any nonsingular matrix. When we use a matrix M in this way, we call it a *preconditioner*. Preconditioners will figure prominently in Sections 7.5 and 7.6. Good splitting matrices make good preconditioners. Verify the following fact: Every iterative method defined by a splitting $A = M - N$ can be viewed as Richardson's method (with $\omega = 1$) applied to the preconditioned system $M^{-1}Ax = M^{-1}b$. This accounts for the importance of Richardson's method. □

Exercise 7.3.23 Damping is a way of taming a nonconvergent iteration to get it to converge. Given a splitting matrix M , which gives the iteration (7.3.21), the corresponding *damped iteration* with *damping factor* $\omega < 1$ is defined by

$$x^{(k+1)} = x^{(k)} + \omega M^{-1}r^{(k)}. \quad (7.3.24)$$

Clearly the splitting matrix of the damped iteration is $\frac{1}{\omega}M$. We have already seen two examples of damped iterations: (i) The damped Jacobi method (Exercise 7.2.28), (ii) Richardson's method.

Show that if $M^{-1}A$ has real, positive eigenvalues, then the damped iteration associated with M converges if ω is sufficiently small. □

Exercise 7.3.25 In principle one can perform iterations (7.3.24) with $\omega > 1$, which would be *extrapolated iterations*. This looks like the same idea as SOR, but it is not. Show that extrapolated Gauss-Seidel is different from SOR. (Note: SOR is a good idea; extrapolated Gauss-Seidel is not.) □

Exercise 7.3.26 In this and the next exercise, you will analyze the convergence of iterative methods by examining the matrix powers G^k . In this exercise we assume G is semisimple. Then G is similar to a diagonal matrix: $G = V\Lambda V^{-1}$, where $\Lambda = \text{diag}\{\lambda_1, \dots, \lambda_n\}$. The main diagonal entries of Λ are the eigenvalues of G (and the columns of V are the corresponding eigenvectors). Show that for all k ,

$$\|G^k\|_2 \leq \kappa_2(V) [\rho(G)]^k.$$

Thus $G^k \rightarrow 0$ linearly if $\rho(G) < 1$. □

Exercise 7.3.27 This exercise studies convergence in the case when the iteration matrix G is defective. Every G has the form $G = XJX^{-1}$, where $J = \text{diag}\{J_1, \dots, J_p\}$ is in Jordan canonical form (see, e.g., [42] or [46]). The *Jordan blocks* J_i are square matrices of various sizes, each having the form

$$J_i = \begin{bmatrix} \lambda & 1 & & \\ & \lambda & 1 & \\ & & \lambda & \ddots \\ & & & \ddots & 1 \\ & & & & \lambda \end{bmatrix},$$

where λ is an eigenvalue of G . Each is upper triangular with λ 's on the main diagonal, 1's on the superdiagonal, and zeros elsewhere.

- (a) Prove that the powers of a Jordan block have the form

$$J_i^k = \begin{bmatrix} \lambda^k & k\lambda^{k-1} & \frac{k(k-1)}{2}\lambda^{k-2} & \cdots \\ & \lambda^k & k\lambda^{k-1} & \ddots \\ & & \lambda^k & \ddots & \frac{k(k-1)}{2}\lambda^{k-2} \\ & & & \ddots & k\lambda^{k-1} \\ & & & & \lambda^k \end{bmatrix}.$$

Specifically, J_i^k has $k+1$ nonzero diagonals (or less, if the dimension of J_i is less), and the j th diagonal from the main diagonal has entries $\binom{k}{j}\lambda^{k-j}$.

- (b) Given a non-negative integer j , define a function p_j by $p_j(k) = \binom{k}{j}$. Show that p_j is a polynomial of degree j .
- (c) Suppose the Jordan block J_i is $m \times m$, and $\lambda \neq 0$. Show that for sufficiently large k , $\|J_i^k\|_\infty = p(k)|\lambda|^k$, where p is a polynomial of degree $m-1$:

$$p(k) = \sum_{j=0}^{m-1} |\lambda|^{-j} \binom{k}{j}.$$

(Recall that $\|B\|_\infty = \max_i \sum_j |b_{ij}|$.)

- (d) Suppose the Jordan block J_i is $m \times m$, and $\lambda = 0$. Show that $J_i^k = 0$ for $k \geq m$.
- (e) Let m be the size of the largest Jordan block of J associated with an eigenvalue λ satisfying $|\lambda| = \rho(G)$. Show that for sufficiently large k , the powers of J satisfy $\|J^k\|_\infty = p(k)\rho(G)^k$, where p is a polynomial of degree $m-1$.

- (f) Show that for large
- k

$$\frac{\|J^{k+1}\|_{\infty}}{\|J^k\|_{\infty}} \approx \rho(G).$$

□

Exercise 7.3.28

- (a) The coefficient matrix for the one-dimensional model problem is the tridiagonal matrix (7.1.5). Show that the associated Jacobi iteration matrix is

$$H = \frac{1}{2} \begin{bmatrix} 0 & 1 & & & \\ 1 & 0 & 1 & & \\ & 1 & 0 & \ddots & \\ & & \ddots & \ddots & 1 \\ & & & 1 & 0 \end{bmatrix}.$$

- (b) We will calculate the spectral radius of
- H
- by finding all of its eigenvalues. The dimension of
- H
- is
- $m-1$
- . Show that
- v
- is an eigenvector of
- H
- with eigenvalue
- λ
- if and only if the difference equation

$$v_{j+1} - 2\lambda v_j + v_{j-1} = 0, \quad j = 1, \dots, m-1, \quad (7.3.29)$$

holds with boundary conditions

$$v_0 = 0 = v_m. \quad (7.3.30)$$

- (c) Since the linear, homogeneous difference equation (7.3.29) is of second order, it has two linearly independent solutions (i.e., it has a two-dimensional solution space). Show that a geometric progression
- $v_j = z^j$
- (with
- $z \neq 0$
-) is a solution of (7.3.29) if and only if
- z
- is a solution of the
- characteristic equation*

$$z^2 - 2\lambda z + 1 = 0. \quad (7.3.31)$$

- (d) This quadratic equation has two solutions
- z_1
- and
- z_2
- . Show that

$$z_1 z_2 = 1 \quad (7.3.32)$$

and

$$\lambda = \frac{1}{2} (z_1 + z_2). \quad (7.3.33)$$

- (e) If the solutions of (7.3.31) satisfy
- $z_1 \neq z_2$
- , the general solution of (7.3.29) is
- $v_j = c_1 z_1^j + c_2 z_2^j$
- , where
- c_1
- and
- c_2
- are arbitrary constants. Show that in this case, any nonzero solution that also satisfies the boundary conditions (7.3.30) must have (i)
- $c_2 = -c_1$
- , (ii)
- $z_1^{2m} = 1$
- , (iii)
- $z_1 = e^{k\pi i/m}$
- , where
- k
- is not a multiple of
- m
- (remember the assumption
- $z_1 \neq z_2$
-). Each choice of
- k
- yields

an eigenvector of H . We can normalize the eigenvector by choosing c_1 . Show that if we take $c_1 = \frac{1}{2i}$ (and hence $c_2 = -\frac{1}{2i}$), then the eigenvector v is given by (iv) $v_j = \sin(jk\pi/m)$.

- (f) Associated with each eigenvector is an eigenvalue. Use (7.3.33) to show that the eigenvalue associated with $z_1 = e^{k\pi i/m}$ is $\lambda_k = \cos(\pi h k)$, where $h = \frac{1}{m}$ is the mesh size. Show that letting $k = 1, \dots, m-1$, we get $m-1$ distinct eigenvalues satisfying $1 > \lambda_1 > \lambda_2 > \dots > \lambda_{m-1} > -1$. Since H has at most $m-1$ eigenvalues, these are all of them. (You can easily check that values of k outside the range $1, \dots, m-1$ and not multiples of m just give these same eigenvalues over and over again.)
- (g) Show that $\lambda_{m-k} = -\lambda_k$ for $k = 1, \dots, m-1$. Conclude that the spectral radius of H is $\cos(\pi h)$.

□

Exercise 7.3.34 This study of the basic properties of Kronecker products of matrices will prepare you for the next exercise. Let X and Y be matrices of dimensions $\alpha \times \beta$ and $\gamma \times \delta$, respectively. Then the *Kronecker product* or *tensor product* of X and Y is the $\alpha\gamma \times \beta\delta$ matrix $X \otimes Y$ defined by

$$X \otimes Y = \begin{bmatrix} x_{11}Y & x_{12}Y & \cdots & x_{1\beta}Y \\ x_{21}Y & x_{22}Y & \cdots & x_{2\beta}Y \\ \vdots & \vdots & \ddots & \vdots \\ x_{\alpha 1}Y & x_{\alpha 2}Y & \cdots & x_{\alpha\beta}Y \end{bmatrix}.$$

- (a) Let X, Y, W , and Z be matrices whose dimensions are such that the products XW and YZ are defined. Show that the product $(X \otimes Y)(W \otimes Z)$ is defined, and $(X \otimes Y)(W \otimes Z) = (XW) \otimes (YZ)$. (Notice the following important special case. If u and v are column vectors such that Xu and Yv are defined, then $u \otimes v$ is a (long) column vector such that $(X \otimes Y)(u \otimes v) = Xu \otimes Yv$.)
- (b) Suppose X is $m \times m$ and Y is $n \times n$. Suppose λ is an eigenvalue of X with associated eigenvector u , and μ is an eigenvalue of Y with associated eigenvector v .
- (i) Show that $\lambda\mu$ is an eigenvalue of $X \otimes Y$ with associated eigenvector $u \otimes v$.
- (ii) Let I_k denote the $k \times k$ identity matrix. Show that $\lambda + \mu$ is an eigenvalue of $X \otimes I_n + I_m \otimes Y$ with associated eigenvector $u \otimes v$.
- (c) Show that if $u_1, \dots, u_\alpha$ is a set of α orthonormal vectors (e.g. eigenvectors), and $v_1, \dots, v_\beta$ is a set of β orthonormal vectors, then $u_i \otimes v_j$, $i = 1, \dots, \alpha$, $j = 1, \dots, \beta$, is a set of $\alpha\beta$ orthonormal vectors.

□

Exercise 7.3.35

- (a) The coefficient matrix A of the two-dimensional model problem (7.1.8) is given by (7.1.10). Show that

$$A = \hat{A} \otimes I_{m-1} + I_{m-1} \otimes \hat{A},$$

where $\hat{A}$ is the one-dimensional model matrix given by (7.1.5).

- (b) Show that the Jacobi iteration matrix associated with (7.1.8) is given by

$$G = \frac{1}{2} (H \otimes I_{m-1} + I_{m-1} \otimes H),$$

where H is as in Exercise 7.3.28.

- (c) Show that the $(m-1)^2$ eigenvalues of G are

$$\lambda_{jk} = \frac{1}{2} (\cos(jh\pi) + \cos(kh\pi)), \quad j, k = 1, \dots, m-1.$$

- (d) Show that $\rho(G) = \cos \pi h$.

□

Exercise 7.3.36 Work out the analogue of Exercise 7.3.35 for the three-dimensional model problem. Conclude that the iteration matrix for Jacobi's method has spectral radius $\cos \pi h$ in this case as well. □

Exercise 7.3.37

- (a) Show that the eigenvalues of the tridiagonal matrix $\hat{A}$ (7.1.5) from the one-dimensional model problem are

$$\lambda_k = 2 - 2 \cos \left(\frac{\pi k}{m} \right) = 4 \sin^2 \left(\frac{\pi k}{2m} \right), \quad k = 1, \dots, m-1.$$

(Hint: This is an easy consequence of results in Exercise 7.3.28. Notice that $\hat{A} = 2(I - H)$.)

- (b) Show that the eigenvalues of the coefficient matrix of the two-dimensional model problem (7.1.8) are

$$\lambda_{jk} = 4 \left[\sin^2 \left(\frac{\pi j}{2m} \right) + \sin^2 \left(\frac{\pi k}{2m} \right) \right] \quad j, k = 1, \dots, m-1.$$

(See Exercise 7.3.35).

- (c) Show that if Richardson's method is applied to the model problem (7.1.8), the optimal ω is $1/4$ (Exercise 7.3.19). Show that with this value of ω , Richardson's method is the same as Jacobi's method (on this particular problem).

- (d) Show that $\kappa_2(A) = \cot^2(\pi h/2)$ for the model problem (7.1.8). Use this figure to calculate the convergence rate for Richardson's method with optimal ω . Show that it agrees with the convergence rate for Jacobi's method calculated in Exercise 7.3.35.

□

Exercise 7.3.38 Consider a system $Ax = b$ that admits a red-black ordering. Thus (after reordering, if necessary)

$$A = \begin{bmatrix} D_1 & B_{12} \\ B_{21} & D_2 \end{bmatrix},$$

where D_1 and D_2 are diagonal matrices.

- (a) Show that the iteration matrix for Jacobi's method applied to this system is

$$G_J = \begin{bmatrix} 0 & C_1 \\ C_2 & 0 \end{bmatrix}, \quad (7.3.39)$$

where $C_1 = -D_1^{-1}B_{12}$ and $C_2 = -D_2^{-1}B_{21}$.

- (b) Matrices of the form (7.3.39) have some interesting spectral properties. Show that if $\begin{bmatrix} x \\ y \end{bmatrix}$ is an eigenvector associated with eigenvalue λ , then $\begin{bmatrix} x \\ -y \end{bmatrix}$ is an eigenvector with eigenvalue $-\lambda$.
- (c) Continuing part (b), show that x (if nonzero) is an eigenvector of C_1C_2 , and y (if nonzero) is an eigenvector of C_2C_1 , with eigenvalue λ^2 .
- (d) Conversely, show that if μ is an eigenvalue of, say, C_2C_1 , then $\pm\sqrt{\mu}$ are eigenvalues of $\begin{bmatrix} 0 & C_1 \\ C_2 & 0 \end{bmatrix}$. (Hint: Construct the eigenvectors.)
- (e) Show that the red-black Gauss-Seidel iteration matrix for A is
- $$G_{GS} = \begin{bmatrix} 0 & C_1 \\ 0 & C_2C_1 \end{bmatrix}.$$
- (f) Show that $\rho(G_{GS}) = \rho(G_J)^2$.

□

Exercise 7.3.40 If the coefficient matrix A is positive definite, it is often advisable to use splittings $A = M - N$ for which M is also positive definite. Show that the following methods have (symmetric and) positive definite splitting matrices, assuming A is positive definite: (a) Richardson, (b) Jacobi, (c) symmetric Gauss-Seidel, (d) SSOR ($0 < \omega < 2$). □

Exercise 7.3.41 Let A be a symmetric matrix, and consider a splitting $A = M - N$. If M is not symmetric, then $A = M^T - N^T$ is another splitting of A . This exercise shows

how to use these two splittings together to build one that is symmetric. Consider an iteration that consists of two half steps, one with splitting matrix M , the other with splitting matrix M^T :

$$Mx^{(k+1/2)} = Nx^{(k)} + b,$$

$$M^Tx^{(k+1)} = N^Tx^{(k+1/2)} + b.$$

- (a) Show that $M^Tx^{(k+1)} = N^TM^{-1}Nx^{(k)} + (I + N^TM^{-1})b$.
- (b) Show that $I + N^TM^{-1} = HM^{-1}$, where $H = M + N^T = M + M^T - A = A + N + N^T$. Show that $N^TM^{-1}N = M^T - HM^{-1}A$.
- (c) Show that if H is nonsingular, then

$$\hat{M}x^{(k+1)} = \hat{N}x^{(k)} + b,$$

where $\hat{M} = MH^{-1}M^T$, and $A = \hat{M} - \hat{N}$. This is the splitting for the combined iteration.

- (d) Show that the splitting matrix $\hat{M}$ is symmetric. Show that $\hat{M}$ is positive definite if H is.
- (e) Show that the SSOR iteration is of this form (assuming A is symmetric). What is H in the SSOR case? Show that if A is positive definite, then H is positive definite if and only if the relaxation factor ω satisfies $0 < \omega < 2$.

□

7.4 DESCENT METHODS; STEEPEST DESCENT

We continue to study the linear system $Ax = b$. From this point on, unless otherwise stated, we shall assume A is symmetric and positive definite. We begin by showing that the problem of solving $Ax = b$ can be reformulated as a minimization problem. We then proceed to a study of methods for solving the minimization problem. Define a function $J : \mathbb{R}^n \rightarrow \mathbb{R}$ by

$$J(y) = \frac{1}{2}y^TAy - y^Tb. \quad (7.4.1)$$

Then the vector that minimizes J is exactly the solution of $Ax = b$.

Theorem 7.4.2 *Let $A \in \mathbb{R}^{n \times n}$ be positive definite, let $b \in \mathbb{R}^n$, and define J as in (7.4.1). Then there is exactly one $x \in \mathbb{R}^n$ for which*

$$J(x) = \min_y J(y),$$

and this x is the solution of $Ax = b$.

Proof. The function J is quadratic in the variables $y_1, \dots, y_n$. We shall discover its minimum by the simple technique of completing the square. Let x denote the solution of $Ax = b$. Then

$$\begin{aligned} J(y) &= \frac{1}{2}y^T Ay - y^T Ax \\ &= \frac{1}{2}y^T Ay - y^T Ax + \frac{1}{2}x^T Ax - \frac{1}{2}x^T Ax \\ &= \frac{1}{2}(y - x)^T A(y - x) - \frac{1}{2}x^T Ax. \end{aligned} \quad (7.4.3)$$

The term $\frac{1}{2}x^T Ax$ is independent of y , so $J(y)$ is minimized exactly when $\frac{1}{2}(y - x)^T A(y - x)$ is minimized. Since A is positive definite, we know that this term is positive unless $y - x = 0$. Thus it takes its minimum value when and only when $y = x$. $\square$

In many problems the function J has physical significance.

Example 7.4.4 In elasticity problems, $J(y)$ denotes the potential energy of the system in configuration y . The configuration that the system actually assumes at equilibrium is the one for which the potential energy is minimized. An example is the simple mass-spring system that was discussed in Example 1.2.10. The matrix A is called the *stiffness matrix*. The term $\frac{1}{2}y^T Ay$ is the *strain energy*, the energy stored in the springs due to stretching or compression. The term $-y^T b$ is the work done by the system against the external forces, which are represented by the vector b . Some details are worked out in Exercise 7.4.18. $\square$

Additional insight is obtained by calculating ∇J , the gradient of J . Recalling that $\nabla J = \left[\frac{\partial J}{\partial y_1}, \dots, \frac{\partial J}{\partial y_n} \right]^T$ and performing the routine computation, we find that

$$\nabla J = Ay - b.$$

This is just the negative of the residual of y as an approximation to the solution of $Ax = b$. Clearly the only point at which the gradient is zero is the solution of $Ax = b$. Thus we see (again) that the only vector that can minimize J is the solution of $Ax = b$.

Descent methods solve $Ax = b$ by minimizing J . These are iterative methods. Each descent method begins with an initial guess $x^{(0)}$ and generates a sequence of iterates $x^{(0)}, x^{(1)}, x^{(2)}, x^{(3)}, \dots$ such that at each step $J(x^{(k+1)}) \leq J(x^{(k)})$, and preferably $J(x^{(k+1)}) < J(x^{(k)})$. In this sense we get closer to the minimum at each step. If at some point we have $Ax^{(k)} = b$ or nearly so, we stop and accept $x^{(k)}$ as the solution. Otherwise we take another step. The step from $x^{(k)}$ to $x^{(k+1)}$ has two ingredients: (i) choice of a search direction, and (ii) a line search in the chosen direction. Choosing a search direction amounts to choosing a vector $p^{(k)}$ that indicates the direction in which we will travel to get from $x^{(k)}$ to $x^{(k+1)}$. Several strategies for choosing $p^{(k)}$ will be discussed below. Once a search direction has been chosen, $x^{(k+1)}$ will be chosen to be a point on the line $\{x^{(k)} + \alpha p^{(k)} \mid \alpha \in \mathbb{R}\}$. Thus we will have

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$$

for some real α_k . The process of choosing α_k from among all $\alpha \in \mathbb{R}$ is the line search. We want to choose α_k in such a way that $J(x^{(k+1)}) \leq J(x^{(k)})$. One way to ensure this is to choose α_k so that $J(x^{(k+1)}) = \min_{\alpha \in \mathbb{R}} J(x^{(k)} + \alpha p^{(k)})$. If α_k is chosen in this way, we say that the line search is *exact*. Otherwise, we say that it is *inexact*.

For some types of functions an exact line search can be a formidable task, but for quadratic functions like (7.4.1), line searches are trivial. The following theorem shows that the correct value of α can be obtained from a formula.

Theorem 7.4.5 Let $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$ be obtained by an exact line search. Then

$$\alpha_k = \frac{p^{(k)T} r^{(k)}}{p^{(k)T} A p^{(k)}},$$

where $r^{(k)} = b - Ax^{(k)}$.

Proof. Let $g(\alpha) = J(x^{(k)} + \alpha p^{(k)})$. The minimizer of g is α_k . A routine computation shows that $g(\alpha) = J(x^{(k)}) - \alpha p^{(k)T} r^{(k)} + \frac{1}{2} \alpha^2 p^{(k)T} A p^{(k)}$. This is a quadratic polynomial in α whose unique minimum can be found by solving the equation $g'(\alpha) = 0$. Since $g'(\alpha) = -p^{(k)T} r^{(k)} + \alpha p^{(k)T} A p^{(k)}$, we have $\alpha^{(k)} = p^{(k)T} r^{(k)} / p^{(k)T} A p^{(k)}$. $\square$

Remark 7.4.6 Notice that $\alpha_k = 0$ if and only if $p^{(k)T} r^{(k)} = 0$. We want to avoid this situation because we would rather have $x^{(k+1)} \neq x^{(k)}$. The equation $p^{(k)T} r^{(k)} = 0$ just says that the search direction is orthogonal to the residual. Thus we would normally want to choose $p^{(k)}$ so that it is not orthogonal to $r^{(k)}$. This is always possible unless $r^{(k)} = 0$ (in which case $x^{(k)}$ is the solution). If we always choose $p^{(k)}$ so that $p^{(k)T} r^{(k)} \neq 0$, we will always have the strict inequality $J(x^{(k+1)}) < J(x^{(k)})$. $\square$

Let us now consider some examples of descent methods.

Exercise 7.4.7 In this exercise the initial guess will be denoted $x^{(1)}$ for notational simplicity. Similarly the first search direction will be denoted $p^{(1)}$. Consider a method in which the first n search directions $p^{(1)}, \dots, p^{(n)}$ are taken to be the standard unit vectors $e_1, \dots, e_n$, the next n search directions $p^{(n+1)}, \dots, p^{(2n)}$ are $e_1, \dots, e_n$ again, so are $p^{(2n+1)}, \dots, p^{(3n)}$, and so on. Suppose an exact line search is performed at each step.

- Show that each group of n steps is one iteration of the Gauss-Seidel method.
- Show that it can happen that $J(x^{(k+1)}) = J(x^{(k)})$ even if $x^{(k)} \neq x$.
- Prove that the situation described in part (b) cannot persist for n consecutive steps. In other words, show that if $x^{(k)} \neq x$, then $J(x^{(k+n)}) < J(x^{(k)})$.

Thus each Gauss-Seidel iteration lowers the potential energy. This idea leads to a proof that the Gauss-Seidel method applied to a positive definite matrix always converges. $\square$

Exercise 7.4.8 Let ω be a fixed number. Consider a method that chooses the search directions as in the previous exercise but takes $x^{(k+1)} = x^{(k)} + \omega \alpha_k p^{(k)}$, where α_k is the increment determined by an exact line search. This method reduces to the previous method if $\omega = 1$. If $\omega \neq 1$, the line search is inexact.

- Show that each group of n steps is an iteration of the SOR method with relaxation factor ω .
- Show that if $p^{(k+1)T} r^{(k)} \neq 0$, then (i) $J(x^{(k+1)}) < J(x^{(k)})$ if $0 < \omega < 2$, (ii) $J(x^{(k+1)}) = J(x^{(k)})$ if $\omega = 0$ or $\omega = 2$, and (iii) $J(x^{(k+1)}) > J(x^{(k)})$ if $\omega < 0$ or $\omega > 2$.
- Show that if $x^{(k)} \neq x$, then (i) $J(x^{(k+n)}) < J(x^{(k)})$ if $0 < \omega < 2$, (ii) $J(x^{(k+n)}) = J(x^{(k)})$ if $\omega = 0$ or $\omega = 2$, and (iii) $J(x^{(k+n)}) > J(x^{(k)})$ if $\omega < 0$ or $\omega > 2$.

This leads to a proof that SOR applied to a positive definite matrix always converges if and only if $0 < \omega < 2$. $\square$

As we have observed previously, SOR can beat Gauss-Seidel significantly if ω is chosen well. This shows that exact line searches are not necessarily better than inexact searches in the long run.

Steepest Descent

The method of *steepest descent* takes $p^{(k)} = r^{(k)}$ and performs exact line searches. Since $r^{(k)} = -\nabla J(x^{(k)})$, the search direction is the direction of steepest descent of J from the point $x^{(k)}$.

To search in the direction of steepest descent is a perfectly natural idea. Unfortunately it doesn't work particularly well. Steepest descent is worth studying, nevertheless, for at least two reasons: (i) It is a good vehicle for introducing the idea of preconditioning. (ii) Minor changes turn the steepest descent algorithm into the powerful conjugate-gradient method.

Example 7.4.9 If we apply the steepest descent method to the same small system as in Example 7.2.3, we obtain

$$x^{(5)} = \begin{bmatrix} 3.9525 \\ 3.0508 \\ 1.9973 \\ 1.0147 \end{bmatrix}, \quad x^{(10)} = \begin{bmatrix} 3.9980 \\ 3.0011 \\ 1.9989 \\ 0.9997 \end{bmatrix}, \quad x^{(15)} = \begin{bmatrix} 3.99993 \\ 3.00008 \\ 1.99999 \\ 1.00002 \end{bmatrix},$$

which is only slightly better than we got using Jacobi's method. After 42 iterations, our approximation agrees with the true solution to twelve decimal places. $\square$

It is a simple matter to program the steepest descent algorithm. Let us consider some of the implementation issues. It will prove worthwhile to begin by writing

down a generic descent algorithm. At each step our approximate solution is updated by

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}. \quad (7.4.10)$$

If we are doing exact line searches, we will have calculated α_k using the formula given in Theorem 7.4.5. This requires, among other things, multiplying the matrix A by the vector $p^{(k)}$. The cost of this operation depends on how sparse A is. In many applications the matrix-vector product is the most expensive step of the algorithm, so we should try not to do too many of them. We also need the residual $r^{(k)} = b - Ax^{(k)}$, which seems to require an additional matrix-vector product $Ax^{(k)}$. We can avoid this by using the simple recursion

$$r^{(k+1)} = r^{(k)} - \alpha_k Ap^{(k)}, \quad (7.4.11)$$

which is an easy consequence of (7.4.10), to update the residual from one iteration to the next. Now the matrix-vector product is $Ap^{(k)}$, which we will have already calculated as part of the computation of α_k .

Exercise 7.4.12 Use (7.4.10) to derive (7.4.11). □

Introducing the auxiliary vector $q^{(k)} = Ap^{(k)}$, we get a generic descent algorithm with exact line search.

Prototype Generic Descent Algorithm (exact line search)

$$\begin{aligned} r^{(0)} &\leftarrow b - Ax^{(0)} \\ p^{(0)} &\leftarrow ? \\ \text{for } k &= 0, 1, 2, \dots \\ &\left[\begin{array}{l} q^{(k)} \leftarrow Ap^{(k)} \\ \alpha_k \leftarrow p^{(k)T} r^{(k)} / p^{(k)T} q^{(k)} \\ x^{(k+1)} \leftarrow x^{(k)} + \alpha^{(k)} p^{(k)} \\ r^{(k+1)} \leftarrow r^{(k)} - \alpha_k q^{(k)} \\ p^{(k+1)} \leftarrow ? \end{array} \right. \end{aligned} \quad (7.4.13)$$

All that is needed is a rule for specifying the search direction. We get steepest descent by setting $p^{(j)} = r^{(j)}$.

In practice we should stop once the iterates have converged to our satisfaction. In Section 7.2 we used a criterion involving the difference between two successive iterates. That criterion can also be used here, but there are other possibilities. For example, the steepest descent algorithm calculates $p^T r = r^T r = \|r\|_2^2$ on each iteration, which makes it easy to use a stopping criterion based on the norm of the residual. Whatever criterion we use, we accept $x^{(k+1)}$ as an adequate approximation of the solution as soon as the criterion is satisfied. We must also realize that if we set a tolerance that is too strict, roundoff errors may prevent termination entirely. It is therefore essential to place a limit l on the number of iterations.

We do not need separate storage locations for $x^{(0)}$, $x^{(1)}$, $x^{(2)}$, and so on; we can have a single vector variable x , which starts out containing the initial guess, carries each of the iterates in turn, and ends up containing the final solution x . Similarly,

a single vector r can be used to store all of the residuals in turn. Initially r can be used to store the right-hand-side vector b , which appears only in the calculation of the initial residual $r^{(0)}$. Similar remarks apply to $p^{(k)}$, $q^{(k)}$, and $\alpha^{(k)}$.

Taking these remarks into account we obtain a more refined statement of the algorithm.

Generic Descent Algorithm (with exact line search) for solving $Ax = b$. On entry the initial guess is stored in x , and the vector b is stored in r . The algorithm returns in x its best estimate of the solution and a flag that indicates whether or not the specified tolerance was achieved.

$$\begin{array}{l}
 r \leftarrow r - Ax \\
 p \leftarrow ? \\
 k \leftarrow 0 \\
 \text{do until satisfied or } k = l \\
 \quad \left[\begin{array}{l}
 q \leftarrow Ap \\
 \alpha \leftarrow p^T r / p^T q \\
 x \leftarrow x + \alpha p \\
 r \leftarrow r - \alpha q \\
 p \leftarrow ? \\
 k \leftarrow k + 1
 \end{array} \right. \quad (7.4.14) \\
 \text{if not satisfied, set flag}
 \end{array}$$

Set $p \leftarrow r$ to get steepest descent.

The matrix A is used to compute the matrix-vector product Ax in the initial residual. Aside from that, it is needed to calculate the matrix-vector product Ap once per iteration. The matrix-vector products are normally performed by a user-supplied subroutine, which accepts a vector p as input and returns the vector Ap . The details of the computation depend on the form of A . If A is a dense matrix stored in the conventional way, one can write a straightforward matrix-vector product algorithm that takes about n^2 flops to do the computation. However, if A is sparse, one should exploit the sparseness to produce Ap in far fewer than n^2 flops.

Suppose, for example, A is the matrix of the model problem (7.1.8). Then each row of A will have at most five nonzero entries, so each element of the vector Ap can be calculated in roughly five flops. Thus the entire vector Ap can be calculated in some $5n$ flops; the cost is $O(n)$ instead of $O(n^2)$.

Let us consider some of the details of applying steepest descent to the model problem (7.1.8). Throughout our discussion of descent algorithms, we have treated the vectors x , r , p , etc. as ordinary column vectors in $\mathbb{R}^n$. For example, $p^T = [p_1, p_2, \dots, p_n]$, and the inner product $p^T r$ in Algorithm 7.4.14 means

$$\sum_{i=1}^n p_i r_i.$$

In the model problem, it is more natural to store the unknown (which we now call u rather than x) in a doubly subscripted array $u_{i,j}$, $i, j = 1, \dots, m-1$. Each entry $u_{i,j}$ represents an approximation to $u(x_i, y_j)$, the value of the solution of a PDE at the grid point (x_i, y_j) . Fortunately there is no need to depart from this notation when we

apply Algorithm 7.4.14 to this problem; we need only interpret the operations in the algorithm appropriately. All of the vectors are stored as doubly subscripted arrays. The update $u \leftarrow u + \alpha p$ means $u_{ij} \leftarrow u_{ij} + \alpha p_{ij}$, $i, j = 1, \dots, m-1$. The inner product $p^T r$ means

$$\sum_{j=1}^{m-1} \sum_{i=1}^{m-1} p_{ij} r_{ij}.$$

The matrix-vector product $q \leftarrow Ap$ is

$$q_{ij} \leftarrow -p_{i-1,j} - p_{i,j-1} + 4p_{i,j} - p_{i,j+1} - p_{i+1,j}, \quad i, j = 1, \dots, m-1. \quad (7.4.15)$$

As usual, the matrix A is not stored in any way, shape, or form. More implementation details are given in Exercise 7.4.19.

Example 7.4.16 We applied steepest descent to the model problem under the same conditions as in Example 7.2.8. The results are summarized in Table 7.6. Comparing

h	Matrix dimension	Iterations to convergence
1/10	81	304
1/20	361	1114
1/40	1521	4010

Table 7.6 Steepest descent method applied to model problem

with the table in Example 7.2.8, we observe that the steepest descent algorithm is just as slow as Jacobi's method. Indeed, the similarity in their performance is striking.

From (7.4.15) we see that cost of the operation $q \leftarrow Ap$ is comparable to that of one Jacobi or Gauss-Seidel iteration. However, steepest descent has the additional inner products $p^T r$ and $p^T q$ and the updates $u \leftarrow u + \alpha p$ and $r \leftarrow r - \alpha q$, each of which takes about $n \approx m^2$ flops. Thus the work to do one steepest descent iteration is somewhat higher than that for a Jacobi or Gauss-Seidel iteration. $\square$

Geometric Interpretation of Steepest Descent

The objective of a descent method is to minimize the function $J(y)$. From (7.4.3) we know that J has the form

$$J(y) = \frac{1}{2}(y - x)^T A(y - x) - \gamma,$$

where x is the solution of $Ax = b$, and γ is constant. Since A is symmetric, there exists an orthogonal matrix U such that $U^T A U$ is a diagonal matrix Λ , by Theorem 5.4.19. The main diagonal entries of Λ are the eigenvalues of A , which are positive. Introducing new coordinates $z = U^T(y - x)$ and dropping the inessential

constant γ and the factor $\frac{1}{2}$, we see that minimizing $J(y)$ is equivalent to minimizing

$$\tilde{J}(z) = z^T \Lambda z = \sum_{i=1}^n \lambda_i z_i^2. \quad (7.4.17)$$

To get a picture of the function $\tilde{J}$, consider the 2×2 case. Now $\tilde{J}$ is a function of two variables, so its contours or level surfaces $\tilde{J}(z_1, z_2) = c$ are curves in the plane. From (7.4.17), the contours have the form

$$\lambda_1 z_1^2 + \lambda_2 z_2^2 = c,$$

which are concentric ellipses centered on the origin. They are ellipses, not hyperbolas, because the eigenvalues λ_1 and λ_2 have the same sign.

The orthogonal coordinate transformation $z = U^T(y - x)$ preserves lengths and angles, so the contours of J are also ellipses of the same shape. For example, the contours of the function $J(y)$ associated with the matrix

$$A = \begin{bmatrix} 6 & 2 \\ 2 & 3 \end{bmatrix}$$

are shown in Figure 7.4. The solution $x = \begin{bmatrix} x_1 & x_2 \end{bmatrix}^T$ lies at the center of the ellipses.

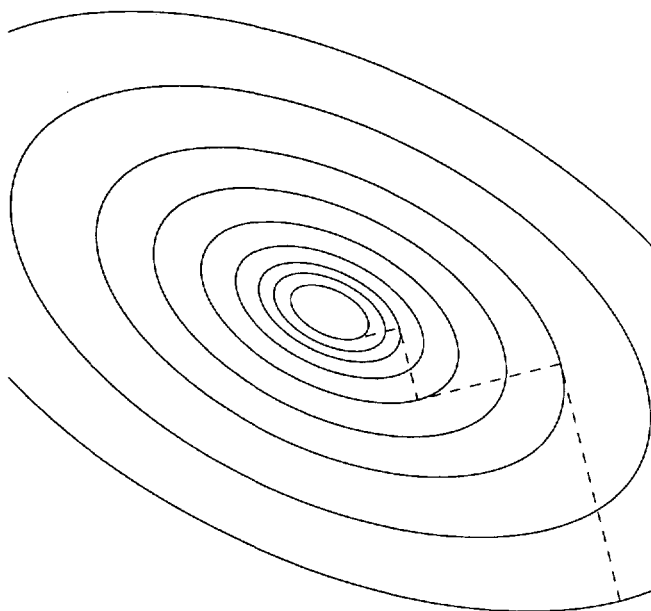


Fig. 7.4 Steepest descent in the 2×2 case

The semiaxes of the ellipses are $\sqrt{c/\lambda_1}$ and $\sqrt{c/\lambda_2}$, whose ratio is $\sqrt{\lambda_2/\lambda_1}$. The eigenvalues of a positive definite matrix are the same as its singular values, so the spectral condition number is equal to the ratio of largest to smallest eigenvalue (see Theorem 4.2.4 and Exercise 5.4.46). Thus the ratio of the semiaxes is $\sqrt{\kappa_2(A)}$. We conclude that the shape of the contours depends on the condition number of A . The greater the condition number, the more eccentric the ellipses are. Figure 7.4 shows the well-conditioned case.

The dotted lines in Figure 7.4 represent four steps of the steepest descent algorithm. From a given point, the search proceeds in the direction of steepest descent, which is orthogonal to the contour line (the direction of no descent). The exact line search follows the search line to the point at which J is minimized. J decreases as long as the search line cuts through the contours. The minimum occurs at the point at which the search line is tangent to a contour. (After that, J begins to increase.) Since the next search direction will be orthogonal to the contour at that point, we see that each search direction is orthogonal to the previous one. Thus the search bounces back and forth in the canyon formed by the function $J(y)$ and proceeds steadily toward the minimum.

The minimum is reached quickly if A is well conditioned. In the best case, $\lambda_1 = \lambda_2$, the contours are circles, the direction of steepest descent (from any starting point) points directly to the center, and the exact minimum is reached in one iteration. If A is well conditioned, the contours will be nearly circular, the direction of steepest descent will point close to the center, and the method will converge rapidly. If, on the other hand, A is somewhat ill conditioned, the contours will be highly eccentric ellipses. From a given point, the direction of steepest descent is likely to point nowhere near the center; it is not a good search direction. In this case the function J forms a steep, narrow canyon. The steepest descent algorithm bounces back and forth in this canyon, taking very short steps, and approaching the minimum with agonizing slowness. This phenomenon does not require extreme ill conditioning. Even if the system is only modestly ill conditioned, and well worth solving from the standpoint of numerical accuracy, the convergence can be very slow.

So far we have been discussing the 2×2 case, but the same difficulties arise in general. In the 3×3 case the level sets are ellipsoids in $\mathbb{R}^3$, and in general they are hyperellipsoids in $\mathbb{R}^n$, whose roundness or lack thereof depends upon $\kappa_2(A)$. Each steepest descent step proceeds in the direction orthogonal to the level hypersurface and continues to a point of tangency with a hypersurface. If the hyperellipsoids are highly eccentric, progress will be slow.

The Jacobi, Gauss-Seidel, and Richardson algorithms all can be viewed as descent algorithms (Exercises 7.4.7 and 7.4.20), and all have difficulty negotiating the steep, narrow canyon. See, for example, Exercise 7.4.22. You could also refer back to Exercise 7.3.19, in which you showed by other means that the convergence of Richardson's method is adversely affected by ill conditioning. Even mild ill conditioning is enough to cause very slow convergence. See Exercise 7.4.24.

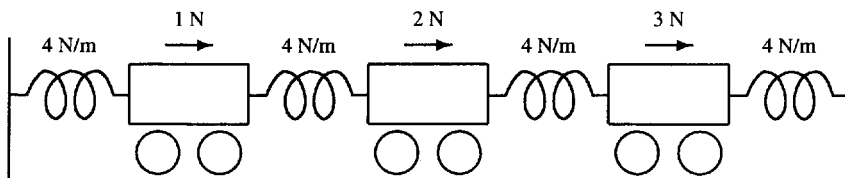


Fig. 7.5 System of three carts

Additional Exercises

Exercise 7.4.18 Suppose a linear spring has stiffness k N/m. Then if we stretch (or compress) it s meters from its equilibrium position, it exerts a restoring force of ks Newtons. To compute the work needed to stretch the spring x meters from its equilibrium position, we integrate the force: $\int_0^x ks \, ds = \frac{1}{2}kx^2$ Joules. The work is equal to the strain energy, the energy stored in the spring due to the deformation. Now consider a system of three masses and four springs, as shown in Figure 7.5. This is the same system as we considered in Example 1.2.10. If external forces are applied to the carts as shown, they will move from their original equilibrium position to a new equilibrium position.

- (a) Each of the springs has stiffness 4 N/m. Show that if the carts 1, 2, and 3 are displaced by y_1 , y_2 , and y_3 meters from their original equilibrium positions (positive displacements being to the right, for example), the total strain energy of the four springs is

$$\frac{4}{2} (y_1^2 + (y_2 - y_1)^2 + (y_3 - y_2)^2 + (-y_3)^2)$$

Joules. Assume that the springs were completely relaxed, neither stretched nor compressed, initially.³

- (b) Show that

$$(y_2 - y_1)^2 = \begin{bmatrix} y_1 & y_2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}.$$

Then show that the total strain energy of the four springs is $\frac{1}{2}y^T Ay$, where A is the same matrix as in Example 1.2.10, and $y = [y_1, y_2, y_3]^T$.

³This assumption simplifies the computation, but it is not essential.

- (c) Show that if external forces are 1, 2 and 3 N to the right, as shown, the potential energy gained due to work against those forces is $-y^T b$ Joules, where $b = [1, 2, 3]^T$.

Thus the total potential energy of the system in configuration y is $J(y) = \frac{1}{2}y^T A y - y^T b$. The actual configuration x that the carts assume in the end is the one for which J is minimized, and that is the one for which $Ax = b$. $\square$

Exercise 7.4.19 Write a program that applies the method of steepest descent to the model problem (7.1.8). Use it to solve the model problem with $f = 0$ and g given by

$$g(x, y) = \begin{cases} 0 & \text{if } x = 0 \\ y & \text{if } x = 1 \\ (x-1) \sin x & \text{if } y = 0 \\ x(2-x) & \text{if } y = 1, \end{cases}$$

as in Example 7.2.8. Use mesh sizes $h = 1/10$, $h = 1/20$, and $h = 1/40$. Starting from $u^{(0)} = 0$, record how many iterations are needed until

$$\|r^{(k+1)}\|_2 < 10^{-8} \|b\|_2.$$

Compare your results with those of Example 7.4.16. Your numbers should be comparable to those given in Example 7.4.16, but, since you are using a different stopping criterion, they will not be exactly the same. Some implementation details follow.

When (7.1.8) is written as a matrix equation $Au = b$, all of the terms associated with the boundary conditions are included in the right-hand-side vector b . Therefore, your program needs to incorporate all of the boundary conditions into b at the start. Consider, for example, the equation associated with the grid point (x_1, y_3) . We have

$$-u_{1,2} - u_{0,3} + 4u_{1,3} - u_{2,3} - u_{1,4} = h^2 f(x_1, y_3).$$

Since $u_{0,3}$ belongs to a boundary grid point, its value is given by the boundary conditions: $u_{0,3} = g(x_0, y_3)$. Since this term is not an unknown, it should be moved to the right-hand side of the equation. Thus the equation takes the form

$$-u_{1,2} + 4u_{1,3} - u_{2,3} - u_{1,4} = h^2 f(x_1, y_3) + g(x_0, y_3),$$

and

$$b_{1,3} = h^2 f(x_1, y_3) + g(x_0, y_3).$$

In this way, each $b_{i,j}$ that is adjacent to a boundary point will include a boundary term. Each corner point (e.g. $b_{1,1}$) will include two boundary terms. (Since our test problem has $f = 0$, all of the nonzero entries in b will be due to boundary terms.) Once the boundary terms have been loaded into b , they can be forgotten; they are not used anywhere else in the program.

The matrix-vector multiply should be effected by a separate subroutine or procedure, the heart of which is a doubly-nested loop like

$$\begin{array}{l} \text{for } j = 1, \dots, m-1 \\ \quad \left[\begin{array}{l} \text{for } i = 1, \dots, m-1 \\ \quad [q_{ij} \leftarrow -p_{i-1,j} - p_{i,j-1} + 4p_{i,j} - p_{i,j+1} - p_{i+1,j}. \end{array} \right. \end{array}$$

If this procedure is to give correct results, the boundary terms must be set to zero. (Since the boundary conditions have been pushed into the vector b , they do not play a role in the matrix A .) If your code sets the boundary values $p_{i,0}$, $p_{i,m}$, $p_{0,j}$, $p_{m,j}$ to zero at the start (and similarly for u) and leaves them zero, the procedure will function correctly.

You might also find it useful to write a separate inner product procedure to calculate the inner products needed for the computation of α . $\square$

Exercise 7.4.20 Show that the steepest descent algorithm is just Richardson's method with variable damping, and Richardson's method is steepest descent with an inexact line search. $\square$

Exercise 7.4.21 Show that the level surfaces of the function $J(y) = \frac{1}{2}y^T Ay - b^T y$ are hyperellipsoids in $\mathbb{R}^n$ with semiaxes $\sqrt{c/\lambda_i}$, $i = 1, \dots, n$, where $\lambda_1, \dots, \lambda_n$ are the eigenvalues of A . $\square$

Exercise 7.4.22 Draw a picture like Figure 7.4 that shows the progress of the Gauss-Seidel method. Draw another picture that shows why Gauss-Seidel converges slowly if A is ill-conditioned. $\square$

Exercise 7.4.23 Draw a picture that suggests that SOR will outperform Gauss-Seidel if ω is chosen well. $\square$

Exercise 7.4.24 Let A be the matrix of the model problem (7.1.8).

(a) Show that

$$\kappa_2(A) = \cot^2(\pi h/2) \approx 2/(\pi h)^2.$$

(See Exercise 7.3.37.)

(b) What relationship do you observe between $\kappa_2(A)$ and known facts about the convergence rates of the Jacobi and Gauss-Seidel methods?

(c) Calculate $\kappa_2(A)$ when $h = 1/40$, and observe that A is only slightly ill conditioned in this case. $\square$

7.5 PRECONDITIONERS

In the preceding section we observed that the surface that represents the function J is a steep, narrow canyon if A is not well conditioned. This causes the steepest descent method (among others) to converge very slowly. A remedy for this problem is to transform the system $Ax = b$ to an equivalent one $\tilde{A}\tilde{x} = \tilde{b}$, for which $\tilde{A}$ is better conditioned than A is. This procedure is called *preconditioning*. We can then apply the descent method to the transformed or *preconditioned* system.

There are many ways to make such a transformation. For example, let M be any simple matrix that approximates A , that is, any splitting matrix, but now we call it a *preconditioner*. Multiply the equation $Ax = b$ by M^{-1} on the left to obtain a transformed system $\tilde{A}\tilde{x} = \tilde{b}$, where $\tilde{A} = M^{-1}A$, $\tilde{x} = x$, and $\tilde{b} = M^{-1}b$. Unfortunately, the resulting $\tilde{A}$ is not symmetric. If M is positive definite, we can make a transformation that preserves the symmetry by making use of the Cholesky decomposition $M = R^T R$. Instead of using $M^{-1}A$, we can use the related matrix $R^{-T}AR^{-1}$. Specifically, we can multiply the equation $Ax = b$ on the left by R^{-T} , and insert the identity matrix $R^{-1}R$ between A and x , to get $(R^{-T}AR^{-1})(Rx) = R^{-T}b$. This is $\tilde{A}\tilde{x} = \tilde{b}$, where $\tilde{A} = R^{-T}AR^{-1}$, $\tilde{x} = Rx$, and $\tilde{b} = R^{-T}b$.

Exercise 7.5.1 Show that $\tilde{A} = R^{-T}AR^{-1}$ is a positive definite matrix, given that A is. In particular, $\tilde{A}$ is symmetric. $\square$

The straightforward approach to preconditioning is simply to apply Algorithm 7.4.14 to the transformed system. Every instance of A , r , x , etc. is replaced by a transformed quantity $\tilde{A}$, $\tilde{r}$, $\tilde{x}$, etc. The algorithm yields $\tilde{x}$ in relatively few iterations, because $\tilde{A}$ is better conditioned than A is. Then we get the solution to our original system by $x = R^{-1}\tilde{x}$.

A more efficient approach is to carry out equivalent operations in the original coordinate system. If this is done, the entire algorithm can be performed without ever generating the transformed quantities. As we shall soon see, the big bonus of this approach is that it makes the computation of R unnecessary. Execution of the descent method will require only M^{-1} .

It is an easy exercise to write down Algorithm 7.4.14 for the transformed system, then translate line by line back into the untransformed coordinate system. We start out with an algorithm in which all of the vectors have tildes on them. Most importantly, $\tilde{A}$ is used in place of A . Then for each line we figure out an equivalent operation in the original coordinate system. For example, the line $\tilde{x} \leftarrow \tilde{x} + \alpha\tilde{p}$ can be transformed to $x \leftarrow x + \alpha p$ by the transformations $x = R^{-1}\tilde{x}$ and $p = R^{-1}\tilde{p}$. (We do not bother to introduce a symbol $\tilde{\alpha}$, because α is a scalar, not a vector. It does not undergo any coordinate transformation.) The transformation $p = R^{-1}\tilde{p}$ serves as the definition of p . It is reasonable to define p by the same transformation law as is used for x , as this allows us to continue to interpret p as the search direction that is taken to get from one iterate to the next. If we define q by $q = R^T\tilde{q}$, the line $\tilde{q} \leftarrow \tilde{A}\tilde{p}$ transforms to $q \leftarrow Ap$. If we want r to denote the residual as it always has up to now, the correct transformation is $r = R^T\tilde{r}$. Then the line $\tilde{r} \leftarrow \tilde{r} - \alpha\tilde{q}$ transforms to $r \leftarrow r - \alpha q$.

Notice also that $\tilde{p}^T \tilde{r} = p^T r$ and $\tilde{p}^T \tilde{q} = p^T q$, so the tildes can be dropped from the computation of α .

So far the transformation process appears quite boring. We started with Algorithm 7.4.14 in the transformed coordinate system, involving $\tilde{p}$, $\tilde{q}$, $\tilde{r}$, etc., then defined p , q , and r in such a way that we could rewrite the equations with the tildes removed. The result is the original Algorithm 7.4.14, or so it appears at first.

The one thing we still need to discuss is the transformation of the lines $\tilde{p} \leftarrow ?$. Now let us focus our attention specifically on the steepest descent method. In the original steepest descent method we had $p = r$, so preconditioned steepest descent will have $\tilde{p} = \tilde{r}$. Now here is the important point. When we transform the equation $\tilde{p} = \tilde{r}$ back to the original coordinate system, we do not get $p = r$, because p and r obey different transformation laws.⁴ Since $\tilde{p} = R p$ and $\tilde{r} = R^{-T} r$, we have $p = R^{-1} R^{-T} r$. Recalling that $M = R^T R$, we see that $p = M^{-1} r$. With this transformation, the symbol R disappears from the algorithm. Thus the preconditioned steepest descent algorithm with preconditioner M is as follows.

Preconditioned Steepest Descent Algorithm (with preconditioner M) for solving $Ax = b$. On entry the initial guess is stored in x , and the vector b is stored in r . The algorithm returns in x its best estimate of the solution and a flag that indicates whether or not the specified tolerance was achieved.

$$\begin{array}{l}
 r \leftarrow r - Ax \\
 p \leftarrow M^{-1} r \\
 k \leftarrow 0 \\
 \text{do until converged or } k = l \\
 \quad \left[\begin{array}{l} q \leftarrow Ap \\ \alpha \leftarrow p^T r / p^T q \\ x \leftarrow x + \alpha p \\ r \leftarrow r - \alpha q \\ p \leftarrow M^{-1} r \\ k \leftarrow k + 1 \end{array} \right. \quad (7.5.2) \\
 \text{if not converged, set flag}
 \end{array}$$

Algorithm 7.5.2 is identical to Algorithm 7.4.14, except for the lines $p \leftarrow M^{-1} r$. Indeed, Algorithm 7.5.2 is an instance of Algorithm 7.4.14. The only effect of applying a preconditioner to the steepest descent algorithm is that the search direction is changed from r to $M^{-1} r$.

Exercise 7.5.3 Check all of the details of the foregoing development. Verify that Algorithm 7.5.2 is indeed the correct translation of the steepest descent algorithm for $\tilde{A}\tilde{x} = \tilde{b}$ where $\tilde{A} = R^{-T} A R^{-1}$. $\square$

⁴There are two transformation laws. In the language of duality, the vectors x and p , which satisfy $z = R^{-1} \tilde{z}$, are *primal*, and the vectors b , r , and q , which satisfy $z = R^T \tilde{z}$, are *dual*.

Examples of Preconditioners

Our derivation assumed that M is positive definite. We thus search for preconditioners among those iterative methods that have positive definite splitting matrices.

Example 7.5.4 Jacobi's method has $M = D$, the diagonal part of A . This M is positive definite if A is. The Jacobi preconditioner, which is also known as the *diagonal preconditioner*, is particularly easy to apply. The operation $p \leftarrow M^{-1}r$ amounts to $p_i \leftarrow r_i/a_{ii}$, $i = 1, \dots, n$. Unfortunately it is not a very powerful preconditioner. $\square$

Exercise 7.5.5 Show that the convergence of the steepest descent algorithm on the model problem (7.1.8) is not accelerated at all by the Jacobi preconditioner. $\square$

In problems for which the main-diagonal entries of the coefficient matrix vary markedly in magnitude, the Jacobi preconditioner is often effective.

Exercise 7.5.6 Apply the steepest descent algorithm to the system

$$\begin{bmatrix} 10 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 13 \\ 4 \end{bmatrix}.$$

Starting with $x^{(0)} = 0$, perform three iterations using (a) no preconditioner, (b) the Jacobi preconditioner. You may do this by hand or, better yet, write a simple MATLAB script. Note the extreme effectiveness of the preconditioner in this case. If you are using MATLAB, you might like to compare the condition numbers of A and $\tilde{A} = D^{-1/2}AD^{-1/2}$. $\square$

On small problems like the one in Exercise 7.5.6, there is no harm in forming the matrix M^{-1} explicitly and using it to perform the preconditioning step $p \leftarrow M^{-1}r$. On big problems it is wasteful to do so. Normally the preconditioning operation is handled by a subroutine that takes r as input and returns p , and normally the subroutine does this without forming M or M^{-1} . For example, a Jacobi preconditioning subroutine will just take each entry of r and divide it by the corresponding main diagonal entry of A .

The procedure for applying the Jacobi preconditioner is perfectly obvious. Now let us consider how to apply a more complicated one. Consider any iterative method that is generated by a splitting matrix M . Recall that the iteration is defined by $Mx^{(k+1)} = Nx^{(k)} + b$, where $A = M - N$. If we perform one iteration to get from $x^{(0)}$ to $x^{(1)}$, we have

$$x^{(1)} = x^{(0)} + M^{-1}r^{(0)}, \quad (7.5.7)$$

where $r^{(0)} = b - Ax^{(0)}$. This was shown in general in Exercise 7.3.20 and for special cases in Exercises 7.2.4, 7.2.12, 7.2.18, 7.2.24, and 7.2.26. For example, if we perform one iteration of SOR, getting from $x^{(0)}$ to $x^{(1)}$, the two iterates are related by (7.7.5.7), where $M = \frac{1}{\omega}D - E$. But the SOR iteration is executed by sweeping through the x vector, correcting the entries one after the other; we never form or even

think about the matrix M or its inverse. If we start from $x^{(0)} = 0$, then $r^{(0)} = b$, and (7.7.5.7) becomes $x^{(1)} = M^{-1}b$. This tells us that when we perform one iteration on the system $Ax = b$, starting with $x^{(0)} = 0$, the result is $M^{-1}b$. (Although we have the SOR example in mind, the conclusion is valid for any iterative method that is generated by a splitting.) For the purpose of preconditioning, we want to compute $M^{-1}r$, not $M^{-1}b$. This can be effected by operating on the system $Ax = r$ instead of $Ax = b$. In summary, to compute $p = M^{-1}r$, apply one iteration, starting from $x^{(0)} = 0$, to the system $Ax = r$. The result is p .

We do not envision using SOR as a preconditioner; its splitting matrix is not symmetric. A better candidate is SSOR.

Example 7.5.8 In the symmetric case, the splitting matrix for SSOR is

$$M = \frac{\omega}{2 - \omega} \left(\frac{1}{\omega} D - E \right) D^{-1} \left(\frac{1}{\omega} D - E^T \right),$$

where D and $-E$ are the diagonal and lower triangular parts of A , respectively (Exercise 7.2.26). As long as $0 < \omega < 2$, and A is positive definite, M is positive definite too (Exercise 7.3.40). This M looks complicated. Fortunately it is easy to apply M^{-1} without actually forming it. We simply perform one SSOR iteration on the system $Ax = r$, starting from initial guess zero. Recall that an SSOR iteration consists of a forward SOR iteration followed by a backward one.

Table 7.7 lists some results from applying steepest descent with the SSOR preconditioner to the model problem (7.1.8) under the same conditions as in Example 7.2.8. We took $h = 1/40$ and tried several values for the relaxation factor ω . The iteration

Preconditioner	ω	Iterations to convergence
none	N/A	4010
SSOR	1.0	575
SSOR	1.3	327
SSOR	1.6	171
SSOR	1.9	58

Table 7.7 Steepest descent method with SSOR preconditioner

count 4010 for no preconditioner is the same as in Example 7.4.16. We observe that the SSOR preconditioner delivers a dramatic improvement, regardless of how ω is chosen. If ω is chosen well, the results are excellent. Comparing with Example 7.2.21, we see that the number of iterations needed here with $\omega = 1.9$ is much less than for SOR with the optimal ω . To be fair, we should point out that the steepest descent iteration with SSOR preconditioner requires several times as much arithmetic as an SOR iteration. Even taking this factor into account, we see that steepest descent with the SSOR preconditioner is competitive with SOR. $\square$

The SSOR preconditioner works quite well and is easy to use, so it will figure prominently in our examples. It is not necessarily the best preconditioner.

Some of the most popular preconditioners (and some of the first to be tried) are the ILU preconditioners, which are based on incomplete LU decompositions. Since we are focusing on the positive definite case, we shall restrict our attention to *incomplete Cholesky* preconditioners. If A is sparse, its Cholesky factor is normally much less sparse. A great deal of fill-in occurs during the elimination process. An incomplete Cholesky decomposition is an approximation $A \approx R^T R$, where R is an upper triangular matrix that is much sparser than the true Cholesky factor. For example, R could have exactly the same sparsity pattern as A . One could produce such an R by carrying out the ordinary Cholesky decomposition algorithm but not allowing any fill-in. Numbers that start out zero stay zero. For a better approximation one can allow partial fill-in. Criteria for deciding which entries are allowed to fill in can be based on element size or element location, for example. A great many strategies have been tried. Once an incomplete Cholesky decomposition has been produced, the product $M = R^T R$ can serve as a preconditioner. Application of the preconditioner requires forward and backward substitution on the extremely sparse triangular factors.

Other preconditioners are based on multigrid and domain decomposition techniques [9], [36], [63]. When applied to the model problem (7.1.8) and a wide variety of similar problems, the best of these are optimal, in the sense that they produce a preconditioned matrix $\tilde{A}$ for which the condition number is bounded: $\kappa_2(\tilde{A}) \leq K$, where K is independent of the mesh size h . It follows that the number of iterations needed to achieve a given accuracy is more or less independent of h .

Additional Exercises

Exercise 7.5.9 Write a program that executes the steepest descent algorithm with the SSOR preconditioner on the model problem (7.1.8). Your preconditioning subroutine should consist of a pair of nested loops, the second of which looks something like this:

```

for j = m - 1, ..., 1
    for i = m - 1, ..., 1
        [  $\hat{p} \leftarrow .25(p_{i,j+1} + p_{i,j-1} + p_{i+1,j} + p_{i-1,j} + r_{i,j})$ 
           $p_{i,j} = p_{i,j} + \omega(\hat{p} - p_{i,j})$  ]
    
```

Before entering the first loop, p should be zero (including boundary values).

To test your program, use the same f , g , and $u^{(0)}$ as in Exercise 7.4.19. Terminate iterations when $p^{(k+1)T} r^{(k+1)} < \epsilon^2 p^{(0)T} r^{(0)}$, where $\epsilon = 10^{-8}$ (see Exercise 7.5.10). With $h = 1/40$, try $\omega = 1.7, 1.75, 1.8, 1.85, 1.9$, and 1.95 . Observe that the preconditioner performs satisfactorily over this fairly wide range of ω values. $\square$

Exercise 7.5.10 Show that if $p = M^{-1}r$, then $p^T r = \|\tilde{r}\|_2^2$, the square of the norm of the residual for the transformed problem. Discuss the pros and cons of a stopping criterion based on $\|\tilde{r}\|$ versus $\|r\|$. $\square$

Exercise 7.5.11 Our development of Algorithm 7.5.2 assumes that the splitting matrix is positive definite. Nevertheless, there is nothing to stop us from running Algorithm 7.5.2 with a nonsymmetric M . In this case we have no justification for the choice of search direction $p = M^{-1}r$, but we are still performing a descent algorithm with exact line searches.

- (a) Try using SOR as a preconditioner for steepest descent on the model problem. How does it compare with SSOR? with no preconditioner?
- (b) Try a preconditioner that does m sweeps of SOR instead of one. Try $m = 2, 3$, and 4 , at least. Notice that as m is increased, the preconditioner gets better. Of course, it also consumes more time.

□

Exercise 7.5.12 Given a preconditioner or splitting matrix M , show that the preconditioned steepest descent algorithm is the same as the iteration generated by the splitting matrix $\frac{1}{\omega}M$, except that the damping parameter varies from one iteration to the next. (See Exercises 7.3.20 and 7.3.23.)

□

Exercise 7.5.13 Let A and M be positive definite matrices, and let $M = R^T R$. Define $A_1 = M^{-1}A$ and $A_2 = R^{-T}AR^{-1}$.

- (a) Show that A_1 and A_2 are similar. That is, $A_2 = S^{-1}A_1S$ for some nonsingular matrix S .
- (b) Show that Richardson's method applied to either A_1 or A_2 will converge if the damping parameter ω is small enough, and the convergence rate is the same for both. (See Exercise 7.3.19.)

□

7.6 THE CONJUGATE-GRADIENT METHOD

All of the iterative methods that we have discussed to this point are limited by their lack of memory. Each uses only information about $x^{(k)}$ to get to $x^{(k+1)}$. All information from earlier iterations is forgotten. The conjugate-gradient (CG) method [39] is a simple variation on steepest descent that performs better because it has a memory.

Our approach will be to introduce the algorithm right away, compare and contrast it with the steepest descent method, and observe how well it performs. Once we have done that, we will derive the algorithm and study its theoretical properties.

We begin with the basic CG algorithm with no preconditioner, which is shown in (7.6.1), below. In appearance this algorithm differs only slightly from steepest descent. The computation of α is organized a bit differently, but this difference is cosmetic. The line searches are still exact; the CG algorithm is an instance of Algorithm 7.4.14. Initially $p \leftarrow r$, so the first step is steepest descent. On subsequent

steps there is a difference. Instead of $p \leftarrow r$, we have $p \leftarrow r + \beta p$. The residual or steepest descent direction still plays an important role in determining the new search direction, but now the old search direction also matters. This is the one point at which memory of past iterations is used. This slight change makes a huge difference.

Conjugate-Gradient Algorithm for solving $Ax = b$. On entry the initial guess is stored in x , and the vector b is stored in r . The algorithm returns in x its best estimate of the solution and a flag that indicates whether or not the specified tolerance was achieved.

$$\begin{aligned}
 & r \leftarrow r - Ax \\
 & p \leftarrow r \\
 & \nu \leftarrow r^T r \\
 & k \leftarrow 0 \\
 & \text{do until converged or } k = l \\
 & \quad \left[\begin{array}{l} q \leftarrow Ap \\ \mu \leftarrow p^T q \\ \alpha \leftarrow \nu / \mu \\ x \leftarrow x + \alpha p \\ r \leftarrow r - \alpha q \\ \nu_+ \leftarrow r^T r \\ \beta \leftarrow \nu_+ / \nu \\ p \leftarrow r + \beta p \\ \nu \leftarrow \nu_+ \\ k \leftarrow k + 1 \end{array} \right. \quad (7.6.1) \\
 & \text{if not converged, set flag}
 \end{aligned}$$

Example 7.6.2 We applied the Conjugate-Gradient algorithm to the model problem under the same conditions as in Example 7.2.8. The results are summarized in Table 7.8. Comparing with the table in Example 7.4.16, we see that the CG algorithm

h	Iterations to convergence
1/10	29
1/20	60
1/40	118

Table 7.8 Conjugate-gradient method (with no preconditioner) applied to model problem

is far superior to steepest descent. Indeed its performance is more in line with that of SOR (Example 7.2.21) or steepest descent preconditioned by SSOR (Example 7.5.8). An advantage of CG over these other two methods is that it does not involve any ω whose optimal value must be guessed. $\square$

Why is CG so much better than steepest descent? To keep the discussion simple, let us make the (inessential) assumption that $x^{(0)} = 0$. Then, after j iterations of the

form $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$, we have

$$x^{(j)} = \alpha_0 p^{(0)} + \alpha_1 p^{(1)} + \dots + \alpha_{j-1} p^{(j-1)}.$$

Thus $x^{(j)}$ lies in the space $\mathcal{S}_j$ spanned by the j search directions $p^{(0)}, \dots, p^{(j-1)}$. This is true for both algorithms; however, they pick out different search directions. Of course, they also compute different coefficients α_k . Interestingly, the two different sets of search directions span the same space. In both cases, $\mathcal{S}_j = \text{span}\{b, Ab, A^2b, \dots, A^{j-1}b\}$, a Krylov subspace. Since steepest descent does exact line searches, $x^{(j)}$ minimizes the energy function J over the j lines $x^{(k)} + \alpha p^{(k)}$, $k = 0, \dots, j-1$. The union of these lines is a tiny subset of $\mathcal{S}_j$. As we shall see, the CG algorithm does much better. By looking backward just slightly, it manages to pick out the $x^{(j)}$ that minimizes J over the entire space $\mathcal{S}_j$.

Preconditioned Conjugate Gradient Algorithm

The CG algorithm is closely related to steepest descent. Its first step is the same as steepest descent, and each subsequent search direction depends in part on the direction of steepest descent. It is therefore reasonable to hope that CG, like steepest descent, can benefit from preconditioning [10].

The procedure for preconditioning CG is the same as for steepest descent. The following algorithm results.

Preconditioned Conjugate-Gradient Algorithm (with preconditioner M) for solving $Ax = b$. On entry the initial guess is stored in x , and the vector b is stored in r . The algorithm returns in x its best estimate of the solution and a flag that indicates whether or not the specified tolerance was achieved.

```

 $r \leftarrow r - Ax$ 
 $s \leftarrow M^{-1}r$ 
 $\nu \leftarrow r^T s$ 
 $p \leftarrow s$ 
 $k \leftarrow 0$ 
do until converged or  $k = l$ 
     $q \leftarrow Ap$ 
     $\mu \leftarrow p^T q$ 
     $\alpha \leftarrow \nu / \mu$ 
     $x \leftarrow x + \alpha p$ 
     $r \leftarrow r - \alpha q$ 
     $s \leftarrow M^{-1}r$ 
     $\nu_+ \leftarrow r^T s$ 
     $\beta \leftarrow \nu_+ / \nu$ 
     $p \leftarrow s + \beta p$ 
     $\nu \leftarrow \nu_+$ 
     $k \leftarrow k + 1$ 
if not converged, set flag

```

(7.6.3)

We derived (7.6.3) by applying CG to the transformed system $\tilde{A}\tilde{x} = \tilde{b}$, where $\tilde{A} = R^{-T}AR^{-1}$, $\tilde{x} = Rx$, $\tilde{b} = R^{-T}b$, and $R^TR = M$. We then translated each expression to an equivalent expression in the original coordinate system. We introduced a new vector $s = M^{-1}r$ for convenience.

The big advantage of transforming back to original coordinate system is that it eliminates the need to calculate R . Only M^{-1} appears in Algorithm 7.6.3.

Exercise 7.6.4 Verify that Algorithm 7.6.3 is indeed the correct translation of the conjugate-gradient algorithm for $\tilde{A}\tilde{x} = \tilde{b}$. $\square$

Algorithm 7.6.3 is yet another instance of Algorithm 7.4.14. In particular, the line searches are exact.

The cost of executing CG is only slightly greater than that of steepest descent. Because the computation of the coefficients α and β has been arranged carefully, only two inner products need to be computed on each iteration, which is the same as for steepest descent. All other costs are virtually the same, except that CG has the additional vector update $p \leftarrow s + \beta p$, which costs $2n$ flops. The storage space required by CG is $5n$, for the vectors x , r , s , p , and q , plus whatever is needed (if any) to store A and M^{-1} . This compares with $4n$ for steepest descent and n for SOR.

Example 7.6.5 Table 7.9 shows the results of applying the CG method with the SSOR preconditioner to the model problem (7.1.8) under the same conditions as in Example 7.2.8. As in Example 7.5.8, we took $h = 1/40$ and tried several values for the relaxation factor ω . The iteration counts are quite low, and they are

Preconditioner	ω	Iterations to convergence
none	N/A	118
SSOR	1.0	49
SSOR	1.3	37
SSOR	1.6	28
SSOR	1.7	26
SSOR	1.8	24
SSOR	1.9	28

Table 7.9 Conjugate-Gradient method with SSOR preconditioner

fairly insensitive to the choice of ω . Comparing with Example 7.2.21, we see that preconditioned CG is clearly superior to SOR. $\square$

In Exercise 7.6.6 you will see that the advantage of preconditioned CG increases as h is made smaller.

Additional Exercises

Exercise 7.6.6 Write a program that applies the conjugate-gradient algorithm with the SSOR preconditioner to the model problem (7.1.8). The instructions in Exercises 7.4.19 and 7.5.9 for executing the steps $p \leftarrow Aq$ and $s \leftarrow M^{-1}r$ are also applicable here. To test your program, use the same f , g , and $u^{(0)}$ as in Exercise 7.4.19. Terminate iterations when $s^{(k+1)T}r^{(k+1)} < \epsilon^2 s^{(0)T}r^{(0)}$, where $\epsilon = 10^{-8}$ (see Exercise 7.5.10).

- Try out your code under the conditions of Example 7.6.5. You should get similar, but not identical, results, since you are using a different stopping criterion.
- Consider the model problem with $h = 1/160$. What is the dimension of the associated matrix? What is its bandwidth (assuming the natural row ordering)? Approximately what fraction of the matrix entries are nonzero?
- With $h = 1/160$, try $\omega = 1.7, 1.75, 1.8, 1.85, 1.9$, and 1.95 . Observe that the preconditioner performs satisfactorily over this range of ω values.
- Apply SOR with $h = 1/160$ for comparison against preconditioned CG. (Use your code from Exercise 7.2.23.) Try $\omega = 1.9, 1.95$, and 1.962 (optimal for $h = 1/160$). Note the sensitivity to the choice of ω . How does SOR compare with CG preconditioned by SSOR?
- Choose a value of ω , say 1.8 or 1.9 , and run your preconditioned CG code with that chosen ω and $h = 1/10, 1/20, 1/40, 1/80$, and $1/160$. Record the number of iterations i_h in each case. Notice that i_h grows as h is decreased, but the growth is moderate. (Optimally we would like to see no growth. There is still room for improvement.)

□

Exercise 7.6.7 Try replacing SSOR by SOR as a preconditioner for CG. How does it work?

□

Exercise 7.6.8 A conjugate-gradient routine `pcg` is supplied with MATLAB. Read about `pcg` in the MATLAB help facilities, and try it out on a discrete Laplacian matrix. Use commands like the following:

```
m = 40;
A = delsq(numgrid('N',m+1));
n = size(A,1)
sol = ones(n,1); % ``all ones'' solution vector.
b = A*sol;        % right-hand side vector.
tol = 1e-12; maxit = 1000;
[x,flag,relres,iter] = pcg(A,b,tol,maxit);
err = norm(sol-x)
```

The matrix A generated here is exactly the matrix of our model problem (7.1.8) with $h = 1/m$. A “nested dissection” ordering is used [30]. The most common way to use `pcg` is to supply the matrix A explicitly in MATLAB’s sparse matrix format, and that is what we are doing here. This is far from the most efficient way to use this matrix or to execute the CG algorithm, but it is convenient and works reasonably well on moderate-sized problems. Here we are running `pcg` with a residual tolerance of 10^{-12} , a maximum of 1000 on the number of iterations, and no preconditioner. Try this out with various values of m , e.g. $m = 40, 80, 160$, and check how many iterations are needed in each case. $\square$

Exercise 7.6.9 MATLAB also provides a routine `cholinc` for computing incomplete Cholesky preconditioners, (see Section 7.5). Read about `cholinc` in the MATLAB help facilities. Repeat Exercise 7.6.8 using an incomplete Cholesky preconditioner $M = R^T R$ based on a drop tolerance of 10^{-2} :

```
droptol = 1e-2;
R = cholinc(A,droptol);
spy(A), spy(R), spy(R'+R),
[x,flag,relres,iter] = pcg(A,b,tol,maxit,R',R);
```

Compare your results with those of Exercise 7.6.8. $\square$

Exercise 7.6.10 Repeat Exercise 7.6.9 with fixed m , say $m = 80$, varying the drop tolerance. Try, for example, $10^0, 10^{-1}, 10^{-2}$, and 10^{-3} . Check the effect on the number of CG iterations needed and the number of nonzeros in the incomplete Cholesky factor. Compare with the number of zeros in the complete Cholesky factor of A , as computed by `chol`. $\square$

Exercise 7.6.11 Repeat Exercises 7.6.8, 7.6.9, and 7.6.10, using Wathen matrices of various sizes. Type `A = gallery('wathen',20,15)`, for example. For a bigger matrix replace the 20 and 15 by larger numbers. Type `help private/wathen` for information about Wathen matrices. $\square$

7.7 DERIVATION OF THE CG ALGORITHM

The conjugate-gradient algorithm is a descent method that performs exact line searches. To complete the formal definition of the algorithm, we need only specify how the search directions are chosen. After some preparation we will be in a position to derive the search directions in a natural way.

In Chapter 3 we introduced the inner product

$$\langle x, y \rangle = \sum_{i=1}^n x_i y_i = y^T x = x^T y \quad (7.7.1)$$

and used it extensively in our discussion of the least-squares problem. It was the only inner product we needed at that time, and we spoke of it as if it were the only inner

product in the world. Now it is time to broaden our view of the concept. Given any positive definite matrix H we can define the *inner product induced by H* by

$$\langle x, y \rangle_H = y^T H x = x^T H y.$$

The inner product (7.7.1), which we will now call the *standard inner product*, is the inner product induced by I .

The inner product induced by H has the same basic algebraic properties as the standard inner product. In particular, $\langle x, x \rangle_H > 0$ if $x \neq 0$. Of course the connection with the Euclidean norm, $\|x\|_2 = \sqrt{\langle x, x \rangle}$, does not hold. Instead we can use the H inner product to induce a different norm

$$\|x\|_H = \sqrt{\langle x, x \rangle_H}.$$

A descent method solves the positive definite system $Ax = b$ by minimizing $J(y) = \frac{1}{2}y^T A y - y^T b$. Using our new notation we can rewrite J as

$$J(y) = \frac{1}{2}\langle y, y \rangle_A - \langle b, y \rangle = \frac{1}{2}\|y\|_A^2 - \langle b, y \rangle.$$

In elasticity problems the term $\frac{1}{2}\langle y, y \rangle_A$ represents the strain energy stored in the deformed elastic structure, so the inner product and norm induced by A are commonly called the *energy inner product* and *energy norm*, respectively.

Thinking of y as an approximation to x , the solution of $Ax = b$, we define the *error* $e = x - y$. Completing the square, as in (7.4.3), we find that

$$J(y) = \frac{1}{2}\|e\|_A^2 - \frac{1}{2}\|x\|_A^2.$$

The energy norm of x is a fixed constant. Thus minimizing J is the same as minimizing the energy norm of the error.

A method that uses exact line searches minimizes the energy norm of the error along a line at each step. This is a one-dimensional minimization. Our objective now is to develop a method that remembers information from past steps so that it can minimize over higher dimensional subspaces. By the j th step, we hope to minimize over a j -dimensional subspace.

Regardless of how we choose the search directions, the following relationships hold. At each step we have $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$. Starting from $x^{(0)}$, j such steps bring us to

$$x^{(j)} = x^{(0)} + \alpha_0 p^{(0)} + \dots + \alpha_{j-1} p^{(j-1)}.$$

At step k the error is $e^{(k)} = x - x^{(k)}$. Clearly the errors satisfy the recursion $e^{(k+1)} = e^{(k)} - \alpha_k p^{(k)}$, and after j steps,

$$e^{(j)} = e^{(0)} - \left(\alpha_0 p^{(0)} + \dots + \alpha_{j-1} p^{(j-1)} \right). \quad (7.7.2)$$

Ideally we would like to have chosen the coefficients $\alpha_0, \dots, \alpha_{j-1}$ so that the energy norm $\|e^{(j)}\|_A$ is as small as possible. By (7.7.2) this is the same as minimizing

$$\|e^{(0)} - \sum_{k=1}^{j-1} \alpha_k p^{(k)}\|_A,$$

which amounts to finding the best approximation to $e^{(0)}$ from the subspace $\mathcal{S}_j = \text{span}\{p^{(0)}, \dots, p^{(j-1)}\}$.

In Chapter 3 we developed the basic characterization theorem (Theorem 3.5.15) for best approximation in Euclidean space. The shortest distance from a point to a line is along the perpendicular. More generally, the best approximation to a vector v from a subspace $\mathcal{S}$ is the unique $s \in \mathcal{S}$ for which the error $v - s$ is orthogonal to every vector in $\mathcal{S}$. In that result, approximations are measured with respect to the Euclidean norm, and orthogonality is reckoned with respect to the standard inner product. In our present scenario we seek a best approximation with respect to the energy norm. Fortunately the basic characterization is valid in this context as well, provided we reckon orthogonality with respect to the energy inner product. Indeed, the theorem is valid for any inner product and its induced norm.

Let H be any positive definite matrix. If $\langle v, w \rangle_H = 0$, we say that the vectors v and w are H -orthogonal and write $v \perp_H w$.

Theorem 7.7.3 *Let $H \in \mathbb{R}^{n \times n}$ be a positive definite matrix, let $v \in \mathbb{R}^n$, and let $\mathcal{S}$ be a subspace of $\mathbb{R}^n$. Then there is a unique $s \in \mathcal{S}$ such that*

$$\|v - s\|_H = \min_{w \in \mathcal{S}} \|v - w\|_H.$$

The vector s is characterized by the condition $v - s \perp_H w$ for all $w \in \mathcal{S}$.

This theorem reduces to Theorem 3.5.15 in the case $H = I$. It can be proved by a straightforward generalization of the proof of Theorem 3.5.15.

Applying Theorem 7.7.3 to our current scenario, we see that $\|e^{(j)}\|_A$ is minimized when $p \in \mathcal{S}_j$ is chosen so that the error $e^{(j)} = e^{(0)} - p$ satisfies

$$e^{(j)} \perp_A p^{(i)} \quad \text{for } i = 0, \dots, j-1. \quad (7.7.4)$$

Two vectors that are orthogonal with respect to the energy inner product are said to be *conjugate*. Our goal now is to develop methods for which the error at each step is conjugate to all of the previous search directions.

The following proposition shows that part of (7.7.4) is achieved by performing exact line searches. As usual $r^{(k+1)}$ denotes the residual $b - Ax^{(k+1)}$.

Proposition 7.7.5 *Let $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$ be obtained from an exact line search. Then $r^{(k+1)} \perp p^{(k)}$ and $e^{(k+1)} \perp_A p^{(k)}$.*

Proof. Successive residuals are related by the recursion $r^{(k+1)} = r^{(k)} - \alpha_k A p^{(k)}$ (which appears as $r \leftarrow r - \alpha q$ in all of our algorithms). Thus

$$\langle r^{(k+1)}, p^{(k)} \rangle = \langle r^{(k)}, p^{(k)} \rangle - \alpha_k \langle A p^{(k)}, p^{(k)} \rangle = 0,$$

since $\alpha_k = \langle r^{(k)}, p^{(k)} \rangle / \langle A p^{(k)}, p^{(k)} \rangle$, by Theorem 7.4.5.

It is easy to check that the error and the residual are connected by the simple equation $A e^{(k+1)} = r^{(k+1)}$. Therefore $\langle e^{(k+1)}, p^{(k)} \rangle_A = \langle A e^{(k+1)}, p^{(k)} \rangle = \langle r^{(k+1)}, p^{(k)} \rangle = 0$. $\square$

From the proof it is clear that Proposition 7.7.5 is basically a restatement of Theorem 7.4.5. Both are special cases of Theorem 7.7.3, as Exercise 7.7.27 shows.

Proposition 7.7.5 is geometrically obvious. The minimum of J on the line $x^{(k)} + \alpha p^{(k)}$ occurs when the directional derivative of J in the search direction is zero. the directional derivative is just the dot product of the gradient with the direction, so the directional derivative is zero exactly when the gradient (in this case, the residual) is orthogonal to the search direction.

According to Proposition 7.7.5, after the first step we have $e^{(1)} \perp_A p^{(0)}$. This is condition (7.7.4) in the case $j = 1$. It is clear from (7.7.4) that we would like to keep all subsequent errors conjugate to $p^{(0)}$. Since the errors are related by the recursion $e^{(k+1)} = e^{(k)} - \alpha_k p^{(k)}$, we can accomplish this by forcing all subsequent search directions to be conjugate to $p^{(0)}$. If we pick $p^{(1)}$ so that $p^{(1)} \perp_A p^{(0)}$ and perform an exact line search, we get an $x^{(2)}$ for which the error satisfies $e^{(2)} \perp_A p^{(1)}$. We thus have $e^{(2)} \perp_A p^{(i)}$ for $i = 0, 1$, which is (7.7.4) for $j = 2$. We can now keep all subsequent errors conjugate to both $p^{(0)}$ and $p^{(1)}$ by making all subsequent search directions conjugate to $p^{(0)}$ and $p^{(1)}$.

By now it is clear that we can achieve (7.7.4) by choosing our search directions in such a way that $p^{(i)} \perp_A p^{(j)}$ for all $i \neq j$ and doing exact line searches. A method with these characteristics is called a *conjugate direction method*.

Theorem 7.7.6 *If a method employs conjugate search directions and performs exact line searches, then the errors $e^{(j)}$ satisfy (7.7.4) for all j . Furthermore*

$$\|e^{(j)}\|_A = \min_{p \in \mathcal{S}_j} \|e^{(0)} - p\|_A,$$

where $\mathcal{S}_j = \text{span}\{p^{(0)}, \dots, p^{(j-1)}\}$.

Proof. We prove (7.7.4) by induction on j . For $j = 1$, (7.7.4) holds because the line search is exact. Now let j be arbitrary and assume we have $e^{(j)} \perp_A p^{(i)}$ for $i = 0, \dots, j-1$. We have to show that $e^{(j+1)} \perp_A p^{(i)}$ for $i = 0, \dots, j$. Since $e^{(j+1)} = e^{(j)} - \alpha_j p^{(j)}$, we have $\langle e^{(j+1)}, p^{(i)} \rangle_A = \langle e^{(j)}, p^{(i)} \rangle_A - \alpha_j \langle p^{(j)}, p^{(i)} \rangle_A = 0 - 0 = 0$ for $i = 0, \dots, j-1$. Thus we need only establish $e^{(j+1)} \perp_A p^{(j)}$. But this is true because the line search on step $j+1$ is exact.

Once we have (7.7.4), the optimality property of $e^{(j)}$ follows from Theorem 7.7.3, as we have already seen. $\square$

Choice of Search Directions

The conjugate gradient method is a conjugate-direction method that chooses search directions by A -orthogonalizing the residual against all previous search directions. The first search direction is $p^{(0)} = r^{(0)}$. Thus the first step of CG is the same as the first step of steepest descent. After k steps with conjugate search directions $p^{(0)}, \dots, p^{(k)}$, the CG algorithm chooses

$$p^{(k+1)} = r^{(k+1)} - \sum_{i=0}^k c_{ki} p^{(i)}, \quad (7.7.7)$$

where the coefficients c_{ki} are chosen so that $p^{(k+1)} \perp_A p^{(i)}$ for $i = 0, \dots, k$. One easily checks that $p^{(k+1)} \perp_A p^{(i)}$ is satisfied if and only if

$$c_{ki} = \frac{\langle r^{(k+1)}, p^{(i)} \rangle_A}{\langle p^{(i)}, p^{(i)} \rangle_A}, \quad i = 0, \dots, k. \quad (7.7.8)$$

This is just the Gram-Schmidt process applied to the energy inner product.

Exercise 7.7.9 Verify that if $p^{(k+1)}$ is given by (7.7.7), then $p^{(k+1)} \perp_A p^{(i)}$ if and only if c_{ki} is given by (7.7.8). $\square$

As long as $r^{(k+1)} \neq 0$, meaning the algorithm has not terminated, (7.7.7) yields a nonzero $p^{(k+1)}$. To see this, note that the connection $Ae^{(k+1)} = r^{(k+1)}$ implies that $\langle e^{(k+1)}, p^{(i)} \rangle_A = \langle r^{(k+1)}, p^{(i)} \rangle$. Therefore (7.7.4), with $j = k + 1$, implies

$$r^{(k+1)} \perp p^{(i)} \quad \text{for } i = 0, \dots, k. \quad (7.7.10)$$

It now follows directly from (7.7.7) that

$$\langle p^{(k+1)}, r^{(k+1)} \rangle = \langle r^{(k+1)}, r^{(k+1)} \rangle = \|r^{(k+1)}\|_2^2. \quad (7.7.11)$$

Thus $\langle p^{(k+1)}, r^{(k+1)} \rangle > 0$ as long as $r^{(k+1)} \neq 0$. This implies that $p^{(k+1)} \neq 0$. It also implies (see Remark 7.4.6) that each CG step results in a strict decrease in energy: $J(x^{(k+1)}) < J(x^{(k)})$.

We will make further use of both (7.7.10) and (7.7.11) later.

Exercise 7.7.12 Verify (7.7.10) and (7.7.11). $\square$

Exercise 7.7.13 Prove that the conjugate gradient algorithm arrives at the exact solution of $Ax = b$ in at most n iterations. $\square$

Efficient Implementation

Now that the search directions have been specified, the description of the CG algorithm is formally complete. However, it remains to be shown that the algorithm can be implemented efficiently. Formulas (7.7.7) and (7.7.8) look like expensive computations, and it appears that all of the previous search directions need to be saved. Fortunately it turns out that all of the c_{ki} are zero, except c_{kk} . Thus only one coefficient needs to be computed by (7.7.8), and only the most recent search direction $p^{(k)}$ needs to be kept for (7.7.7), which turns into the elegant statement $p \leftarrow r + \beta p$ in Algorithm 7.6.1.

Proving that the c_{ki} are zero requires some preparation. We begin by recalling some terminology that we introduced in Section 6.3. Given a vector v and a positive integer j , the *Krylov subspace* $\mathcal{K}_j(A, v)$ is defined by

$$\mathcal{K}_j(A, v) = \text{span}\{v, Av, A^2v, \dots, A^{j-1}v\}.$$